

Módulo 2 - Práctica

Curso Spring Boot 2026

Índice

Módulo 2 - Práctica acumulativa	5
Formas de trabajar con el proyecto	5
Desde consola y editor de texto	5
Desde IntelliJ IDEA	5
Desde Eclipse	5
Desde VS Code	5
Punto 2.1 - Capas de la aplicación: Controlador	5
Paso 1 - Repasar el estado actual	5
Pregunta	6
Respuesta razonada	6
Paso 2 - Identificar la lógica que debería ir en un servicio	6
Pregunta	6
Respuesta razonada	6
Paso 3 - Refactorizar el método crear	7
Pregunta	7
Respuesta razonada	7
Paso 4 - Añadir un endpoint con @RequestHeader	7
Pregunta	8
Respuesta razonada	8
Paso 5 - Añadir un endpoint con @RequestParam y valores por defecto	8
Sin parámetros: primera página de 20	8
Página 0, tamaño 1	8
Página 1, tamaño 1	9
Pregunta	9
Respuesta razonada	9
Paso 6 - Añadir un endpoint que devuelve 204	9
Pregunta	9
Respuesta razonada	9
Paso 7 - Refactorizar el controlador para que sea más fino	9
Pregunta	10
Respuesta razonada	10
Paso 8 - Probar el controlador refactorizado	10
Listar	10
Consultar	10
Crear	10
Actualizar	11
Actualizar parcial	11

Eliminar	11
Info petición	11
Pregunta	11
Respuesta razonada	11
Paso 9 - Analizar los códigos de estado devueltos	11
Paso 10 - Probar un error de validación	11
Pregunta	12
Respuesta razonada	12
Paso 11 - Errores comunes del ejercicio	12
Paso 12 - Reto resuelto - Endpoint que devuelve un alumno con enlaces HATEOAS	12
Pregunta	13
Respuesta razonada	13
Conclusión y enlace al siguiente punto	13
Punto 2.2 - Capas de la aplicación: Servicio	14
Paso 1 - Repasar el estado actual	14
Pregunta	14
Respuesta razonada	14
Paso 2 - Crear el paquete service	14
Pregunta	14
Respuesta razonada	14
Paso 3 - Crear la excepción de negocio	14
Pregunta	15
Respuesta razonada	15
Paso 4 - Crear el AlumnoService	15
Pregunta	16
Respuesta razonada	16
Paso 5 - Refactorizar el controlador para que use el servicio	16
Pregunta	18
Respuesta razonada	18
Paso 6 - Arrancar y probar el CRUD completo	18
Listar	18
Consultar	18
Crear	18
Actualizar	18
Actualizar parcial	19
Eliminar	19
Pregunta	19
Respuesta razonada	19
Paso 7 - Probar el error de negocio	19
Pregunta	19
Respuesta razonada	19
Paso 8 - Probar un error técnico	19
Pregunta	20
Respuesta razonada	20
Paso 9 - Analizar la separación de capas	20
Pregunta	20
Respuesta razonada	20
Paso 10 - Errores comunes del ejercicio	20
Paso 11 - Reto resuelto - Servicio para el recurso Expediente	20
Pregunta	22
Respuesta razonada	22

Conclusión y enlace al siguiente punto	23
Punto 2.3 - Capas de la aplicación: Repositorio	23
Contexto del ejercicio	23
Requisitos previos	23
Paso 1 - Repasar el estado actual	23
Pregunta	24
Respuesta razonada	24
Paso 2 - Crear el paquete repository	24
Desde IntelliJ IDEA	24
Desde Eclipse	24
Desde VS Code	24
Desde consola	24
Paso 3 - Crear AlumnoRepository	24
Qué debes entender del código	26
Paso 4 - Refactorizar AlumnoService	27
Pregunta	28
Respuesta razonada	28
Paso 5 - Arrancar y probar el CRUD completo	29
Prueba importante de generación de ID	29
Paso 6 - Analizar la separación de capas	29
Paso 7 - Probar el aislamiento del repositorio	30
Pregunta	30
Respuesta razonada	30
Paso 8 - Probar accesos concurrentes	31
Qué demuestra y qué no demuestra	31
Paso 9 - Errores comunes del ejercicio	31
Paso 10 - Reto resuelto: repositorio para Expediente	32
10.1 Crear ExpedienteRepository	32
10.2 Refactorizar ExpedienteService	33
10.3 Regresión del recurso	34
Pregunta final	34
Respuesta razonada	34
Comprobación final de 2.3	34
Punto 2.4 - Inyección de dependencias y configuración	35
Contexto del ejercicio	35
Paso 1 - Repasar el estado actual	35
Pregunta	35
Respuesta razonada	35
Paso 2 - Crear el paquete config	35
Paso 3 - Crear DatosInicialesConfig	36
Pregunta	36
Respuesta razonada	37
Paso 4 - Limpiar los constructores de los repositorios	37
Comprobación importante	37
Paso 5 - Añadir propiedades personalizadas	37
Paso 6 - Leer propiedades con @Value	37
Pregunta	38
Respuesta razonada	38
Paso 7 - Exponer propiedades no sensibles en un endpoint	38
Paso 8 - Crear el perfil de desarrollo	39
Paso 9 - Crear el perfil de producción	39
Paso 10 - Probar CommandLineRunner	40
Pregunta	40
Respuesta razonada	40
Paso 11 - Errores comunes del ejercicio	40
Paso 12 - Reto resuelto: bean personalizado con @Bean	41

Por qué esta variante	41
Comprobaciones adicionales de 2.4	42
Verificar el perfil desde línea de comandos	42
Añadir un perfil de test	42
Comprobar que no hay construcción manual de beans	42
Verificar el runner sin depender del orden del mapa	42
Punto 2.5 - Introducción al testing	43
Contexto del ejercicio	43
Paso 1 - Verificar la dependencia de test	43
Pregunta	43
Respuesta razonada	43
Paso 2 - Crear la estructura de paquetes de test	43
Paso 3 - Crear el setup de AlumnoServiceTest	43
Paso 4 - Test de consulta exitosa	44
Paso 5 - Test de consulta no encontrada	44
Paso 6 - Test de DNI duplicado	45
Paso 7 - Test de creación exitosa	45
Paso 8 - Ejecutar los tests del servicio	46
Pregunta	46
Respuesta razonada	46
Paso 9 - Crear AlumnoControllerTest	46
Nota 2026: @MockBean	47
Paso 10 - Añadir test de POST	47
Paso 11 - Errores comunes del ejercicio	48
Paso 12 - Reto resuelto: ExpedienteServiceTest	49
Extensión práctica - completar la red de seguridad	50
Test directo de AlumnoRepository	50
Completar ExpedienteServiceTest con código ejecutable	50
Añadir casos web de error	51
Comprobar independencia	52
Punto 2.6 - Buenas prácticas de organización, calidad y documentación	52
Contexto del ejercicio	52
Paso 1 - Revisar la estructura actual	52
Pregunta	53
Respuesta razonada	53
Paso 2 - Identificar problemas de nomenclatura	53
Paso 3 - Añadir Javadoc a las clases principales	53
Paso 4 - Añadir Javadoc a métodos públicos	53
Paso 5 - Reorganizar los paquetes por funcionalidad	54
Regla de seguridad	54
Pregunta	55
Respuesta razonada	55
Paso 6 - Añadir Checkstyle a Maven	55
Paso 7 - Ajustar la configuración de Checkstyle	55
Paso 8 - Añadir OpenAPI	56
Paso 9 - Añadir anotaciones OpenAPI	56
Paso 10 - Crear el README del proyecto	57
Ejecutar	57
Perfiles	57
Endpoints principales	57
Documentación API	57
Arquitectura	58
Paso 12 - Reto resuelto: common/util y FechasUtil	58
Por qué constructor privado	59
Usar la utilidad sin contaminar el contrato JSON	59
Comprobaciones adicionales de cierre	59

Añadir plugin de Javadoc si el build lo necesita explícitamente	59
Verificar OpenAPI automáticamente	60
Verificar Swagger UI	60
Gate manual final del alumno	60
Opcional: inspeccionar SpotBugs sin convertirlo todavía en gate duro	60
Comprobación del README desde cero	60

Módulo 2 - Práctica acumulativa

Esta guía parte del proyecto final publicado de M1 y evoluciona el mismo proyecto de forma acumulativa. No reconstruyas ejemplos aislados: cada punto conserva los contratos externos ya aprendidos salvo cuando el propio paso indique una evolución deliberada.

Formas de trabajar con el proyecto

Desde consola y editor de texto

Usa el Maven Wrapper incluido (`./mvnw` o `mvnw.cmd`) y un editor de texto. Los comandos de esta guía son reproducibles sin depender de un IDE.

Desde IntelliJ IDEA

Abre `M2/proyecto/pom.xml` como proyecto Maven, usa Java 17 y ejecuta `MiProyectoApplication` o los objetivos Maven indicados en cada paso.

Desde Eclipse

Importa `M2/proyecto` como **Existing Maven Project**, selecciona Java 17 y utiliza `Run As / Maven build` o `Spring Boot App` según corresponda.

Desde VS Code

Abre `M2/proyecto`, instala las extensiones Java/Spring si las utilizas y ejecuta el Maven Wrapper desde la terminal integrada.

Punto 2.1 - Capas de la aplicación: Controlador

Contexto del ejercicio: Vamos a refactorizar el `AlumnoController` que construimos en el Módulo 1 para aplicar las buenas prácticas de esta sesión. El controlador funcionaba, pero tenía un problema: la lógica estaba mezclada con el manejo HTTP. Vamos a limpiarlo y a añadir algunas funcionalidades nuevas usando las anotaciones que hemos visto. Requisitos previos: Tener el proyecto `mi-proyecto` con el `AlumnoController` y el `AlumnoDTO` del Módulo 1.

Nota 2026 - contrato acumulativo. Los ejemplos de este punto se estudian sobre el CRUD ya construido en M1. La implementación final conserva filtro curso, orden sort, paginación `page/size`, generación de ID por máximo numérico + 1, 201 Created con Location, /info-peticion, /promocionar y los enlaces del reto. Los helpers internos son un paso pedagógico: en 2.2 la lógica abandona definitivamente el controlador.

Paso 1 - Repasar el estado actual

Abre el `AlumnoController` y observa su contenido. Debe tener: Una lista de alumnos en memoria.

- Un método listar con GET.
- Un método consultar con GET y `@PathVariable`.

- Un método crear con POST y @RequestBody.
- Un método actualizar con PUT.
- Un método actualizarParcial con PATCH.
- Un método eliminar con DELETE.
- Arranca la aplicación y verifica que todo funciona. Si algo no funciona, repasa el Módulo 1.

Pregunta

¿Ves lógica de negocio mezclada con el manejo HTTP en algún método?

Respuesta razonada

Sí, cuando el controlador busca, genera IDs, valida duplicados o modifica colecciones está realizando trabajo que debe migrar a servicio/repositorio. El manejo HTTP sí permanece en el controlador.

Paso 2 - Identificar la lógica que debería ir en un servicio

Vamos a analizar el método crear para ver qué lógica contiene:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    if (dto.getIdificador() == null || dto.getIdificador().isBlank()) {
        dto.setIdificador(String.valueOf(alumnos.stream()
            .map(AlumnoDTO::getIdificador)
            .filter(id -> id != null && id.matches("\\d+"))
            .mapToInt(Integer::parseInt)
            .max()
            .orElse(0) + 1));
    }
    alumnos.add(dto);
    return ResponseEntity.status(HttpStatus.CREATED).body(dto);
}
```

Hay tres cosas aquí:

1. Generación de ID. Es una regla de negocio: cómo se asigna un identificador a un alumno nuevo.
2. Persistencia. Añadir el alumno a la lista.
3. Construcción de la respuesta HTTP. Devolver 201 con el alumno.

Las dos primeras son lógica de negocio; la tercera es responsabilidad del controlador. En un diseño limpio, las dos primeras irían en un servicio. Como en este punto todavía no hemos visto la capa de servicio (la veremos en el punto 2.2), vamos a hacer una refactorización parcial: extraer la lógica a métodos privados del controlador. No es la solución final, pero nos permite ver la diferencia entre “lógica” y “manejo HTTP”.

Pregunta

¿Qué ventaja tiene separar la lógica de negocio del manejo HTTP?

Respuesta razonada

Permite cambiar reglas, almacenamiento o canales de entrada de forma independiente y facilita pruebas unitarias de cada responsabilidad.

Paso 3 - Refactorizar el método crear

Vamos a extraer la lógica a un método privado guardarAlumno:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    AlumnoDTO creado = guardarAlumno(dto);
    URI location = URI.create("/api/v1/alumnos/" + creado.getIdentificador());
    return ResponseEntity.created(location).body(creado);
}

private AlumnoDTO guardarAlumno(AlumnoDTO dto) {
    if (dto.getIdentificador() == null || dto.getIdentificador().isBlank()) {
        dto.setIdentificador(String.valueOf(alumnos.stream()
            .map(AlumnoDTO::getIdentificador)
            .filter(id -> id != null && id.matches("\\d+"))
            .mapToInt(Integer::parseInt)
            .max()
            .orElse(0) + 1));
    }
    alumnos.add(dto);
    return dto;
}
```

El método crear ahora solo hace tres cosas: delega en guardarAlumno, construye la URI del recurso creado y devuelve 201 con la cabecera Location. El método guardarAlumno contiene la lógica de generación de ID y persistencia. Fíjate en la cabecera Location: es una buena práctica que permite al cliente saber dónde consultar el recurso creado. Reinicia y prueba:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"22222222D","fechaNacimiento":"2010-07-15","cur
```

Verás un 201 con la cabecera Location: /api/v1/alumnos/3.

Pregunta

¿Qué ventaja tiene que el cliente reciba la URL del recurso creado?

Respuesta razonada

Permite al cliente localizar inmediatamente el nuevo recurso mediante la cabecera Location, que completa la semántica de 201 Created.

Paso 4 - Añadir un endpoint con @RequestHeader

Vamos a añadir un endpoint que devuelva información sobre la petición: el User-Agent y el idioma preferido. Es un ejemplo de cómo leer cabeceras con @RequestHeader. Añade el método:

```
import org.springframework.web.bind.annotation.RequestHeader;
@GetMapping("/info-peticion")
public Map<String, String> infoPeticion(
    @RequestHeader(value = "User-Agent", required = false) String userAgent,
    @RequestHeader(value = "Accept-Language", required = false) String idioma) {
    return Map.of(
        "userAgent", userAgent != null ? userAgent : "desconocido",
        "idioma", idioma != null ? idioma : "desconocido"
    );
}
```

Este método lee dos cabeceras: User-Agent y Accept-Language. Si no vienen, usa “desconocido”. Devuelve un Map que Spring Boot serializa a JSON. Reinicia y prueba:

```
curl -i http://localhost:8080/api/v1/alumnos/info-peticion \
-H "User-Agent: MiCliente/1.0" \
-H "Accept-Language: es-ES"
```

Verás algo como:

```
{
  "userAgent": "MiCliente/1.0",
  "idioma": "es-ES"
}
```

Si no envías las cabeceras, verás “desconocido” en ambos campos.

Pregunta

¿Para qué sirve leer el User-Agent en un controlador? ¿Qué información aporta?

Respuesta razonada

Aporta información sobre el cliente que hizo la petición. Puede servir para observabilidad o compatibilidad, pero no debe considerarse una identidad fiable ni una frontera de seguridad.

Paso 5 - Añadir un endpoint con @RequestParam y valores por defecto

Vamos a modificar el endpoint de listado para que acepte parámetros de paginación con valores por defecto. Modifica el método listar:

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestParam(required = false) String curso,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size) {
    var stream = alumnos.stream();
    if (curso != null && !curso.isBlank()) {
        stream = stream.filter(a -> a.getCurso().equalsIgnoreCase(curso));
    }
    return stream
        .skip((long) page * size)
        .limit(size)
        .toList();
}
```

Lo que hace: curso es opcional. Si no se envía, no filtra.

- page tiene valor por defecto 0.
- size tiene valor por defecto 20.
- El stream salta los primeros page * size elementos y limita a size.
- Reinicia y prueba:

Sin parámetros: primera página de 20

```
curl http://localhost:8080/api/v1/alumnos
```

Página 0, tamaño 1

```
curl "http://localhost:8080/api/v1/alumnos?page=0&size=1"
```

Página 1, tamaño 1

```
curl "http://localhost:8080/api/v1/alumnos?page=1&size=1"
```

Observa que cada página devuelve un subconjunto distinto.

Pregunta

¿Qué ventaja tiene paginar las colecciones? ¿Qué problema evita cuando hay muchos elementos?

Respuesta razonada

Evita devolver volúmenes crecientes de datos en una sola respuesta, controla memoria/ancho de banda y mantiene tiempos de respuesta previsibles.

Paso 6 - Añadir un endpoint que devuelve 204

Vamos a añadir un endpoint que no devuelva cuerpo. Ya tenemos el DELETE, pero vamos a añadir otro para ver la diferencia entre 200 y 204. Añade un método que marque a todos los alumnos como “promocionados” (sin devolver cuerpo):

```
@PostMapping("/promocionar")
public ResponseEntity<Void> promocionar() {
    alumnos.forEach(a -> a.setCurso(a.getCurso() + " (promocionado)"));
    return ResponseEntity.noContent().build();
}
```

Este método modifica el estado de todos los alumnos y devuelve 204 No Content, porque no hay nada que devolver. El cliente sabe que la operación ha ido bien porque el código es 204. Reinicia y prueba:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos/promocionar
```

Verás un 204 sin cuerpo. Luego consulta la lista y verás que los cursos han cambiado.

Pregunta

¿Por qué el 204 no lleva cuerpo? ¿Qué ventaja tiene frente a devolver 200 con un mensaje?

Respuesta razonada

Porque 204 No Content comunica que la operación terminó correctamente y que no hay representación que devolver; evita un cuerpo artificial sin valor semántico.

Paso 7 - Refactorizar el controlador para que sea más fino

Ahora que hemos añadido varios métodos, vamos a revisar el controlador completo y ver si hay lógica que podamos extraer a métodos privados. El objetivo es que cada método público tenga 5-10 líneas como máximo. Por ejemplo, el método actualizar:

```
@PutMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizar(
    @PathVariable String id,
    @RequestBody AlumnoDTO dto) {
    for (int i = 0; i < alumnos.size(); i++) {
        if (alumnos.get(i).getIdentificador().equals(id)) {
            dto.setIdentificador(id);
            alumnos.set(i, dto);
            return ResponseEntity.ok(dto);
        }
    }
}
```

```
    return ResponseEntity.notFound().build();
}
```

Tiene 10 líneas. Podemos extraer la búsqueda a un método privado:

```
@PutMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizar(
    @PathVariable String id,
    @RequestBody AlumnoDTO dto) {
    return buscarPorId(id)
        .map(existente -> {
            dto.setIdentificador(id);
            alumnos.set(alumnos.indexOf(existente), dto);
            return ResponseEntity.ok(dto);
        })
        .orElse(ResponseEntity.notFound().build());
}
private Optional<AlumnoDTO> buscarPorId(String id) {
    return alumnos.stream()
        .filter(a -> a.getIdentificador().equals(id))
        .findFirst();
}
```

Ahora el método actualizar es más declarativo y el método buscarPorId se puede reutilizar en otros métodos.

Pregunta

¿Qué ventaja tiene extraer la búsqueda a un método privado?

Respuesta razonada

Reduce duplicación y hace visible una operación con nombre propio. En 2.2 esa lógica deja de ser un helper del controlador y se mueve al servicio.

Paso 8 - Probar el controlador refactorizado

Reinicia la aplicación y prueba todos los endpoints:

Listar

```
curl http://localhost:8080/api/v1/alumnos
```

Consultar

```
curl http://localhost:8080/api/v1/alumnos/1
```

Crear

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"María","apellidos":"López","dni":"33333333E","fechaNacimiento":"2011-03-20","curso":1}'
```

Actualizar

```
curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{"nombre":"Ana","apellidos":"García López","dni":"12345678A","fechaNacimiento":"2010-05-12",'
```

Actualizar parcial

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{"curso": "6º Primaria B"}'
```

Eliminar

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/2
```

Info petición

```
curl http://localhost:8080/api/v1/alumnos/info-peticion
```

Todos los endpoints deben seguir funcionando. La refactorización no debe romper nada.

Pregunta

¿Qué ventaja tiene refactorizar el controlador antes de añadir más funcionalidades?

Respuesta razonada

Reduce deuda antes de añadir nuevas capacidades: cada funcionalidad nueva entra ya en una estructura con límites más claros.

Paso 9 - Analizar los códigos de estado devueltos

A medida que has probado los endpoints, has visto distintos códigos de estado. Vamos a recopilarlos: Operación Código Cabecera especial GET colección 200 — GET individual existente 200 — GET individual inexistente 404 — POST crear 201

Location

POST promocionar 204 — PUT actualizar existente 200 — PUT actualizar inexistente 404 — PATCH actualizar existente 200 — PATCH actualizar inexistente 404 — DELETE existente 204 — Operación Código Cabecera especial DELETE inexistente 404 — Cada operación devuelve el código que le corresponde. Esa es la base de una API predecible.

Paso 10 - Probar un error de validación

Vamos a provocar un error a propósito. Envía un POST con un JSON mal formado:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro", "apellidos":"Sánchez",}'
```

Verás un 400 Bad Request con un mensaje que describe el error de sintaxis. Spring Boot captura la excepción de Jackson y devuelve un JSON con el detalle.

Pregunta

¿Qué otros errores de formato crees que provocarían un 400?

Respuesta razonada

Por ejemplo JSON con comas sobrantes, llaves sin cerrar, tipos incompatibles o fechas que no respetan el patrón esperado pueden impedir la deserialización y producir 400.

Paso 11 - Errores comunes del ejercicio

Error Causa Solución Método HTTP incorrecto Usar el método correcto

405 Method Not Allowed

Añadir -H "Content-Type: Falta Content-Type

415 Unsupported Media Type
application/json"

JSON mal formado Revisar la sintaxis

400 Bad Request

Usar required = false o Un parámetro es null

NullPointerException

valores por defecto La lista es inmutable Usar new ArrayList<>(...)

UnsupportedOperationException

El Location no aparece

Falta ResponseEntity.created(location)

Usarlo El 204 devuelve cuerpo Se usa ResponseEntity.ok() Usar noContent().build() El PATCH no actualiza El campo no está en el Map Verificar el nombre

Paso 12 - Reto resuelto - Endpoint que devuelve un alumno con enlaces HATEOAS

Reto: Añadir un endpoint que devuelva un alumno junto con enlaces a otros recursos relacionados (documentos, curso, etc.). Es una

aproximación sencilla a HATEOAS (Hypermedia as the Engine of Application State).

Solución paso a paso:

Paso 1: Añadir el método al controlador:

```

@GetMapping("/{id}/con-enlaces")
public ResponseEntity<Map<String, Object>> consultarConEnlaces(@PathVariable String id) {
    return buscarPorId(id)
        .map(alumno -> {
            Map<String, Object> respuesta = new LinkedHashMap<>();
            respuesta.put("alumno", alumno);
            respuesta.put("_links", Map.of(
                "self", "/api/v1/alumnos/" + id,
                "documentos", "/api/v1/alumnos/" + id + "/documentos",
                "curso", "/api/v1/cursos/" + alumno.getCurso()
            ));
            return ResponseEntity.ok(respuesta);
        })
}

```

```
        .orElse(ResponseEntity.notFound().build());  
    }
```

Lo que hace:

1. Busca el alumno por ID.
2. Si lo encuentra, construye un Map con dos claves: alumno y _links.
3. _links contiene URLs a otros recursos relacionados.
4. Devuelve 200 con el Map.

Paso 2: Reiniciar y probar:

```
curl http://localhost:8080/api/v1/alumnos/1/con-enlaces
```

Verás algo como:

```
{  
  "alumno": {  
    "id": "1",  
    "nombre": "Ana",  
    "apellidos": "García López",  
    "dni": "12345678A",  
    "fechaNacimiento": "2010-05-12",  
    "curso": "5º Primaria"  
  },  
  "_links": {  
    "self": "/api/v1/alumnos/1",  
    "documentos": "/api/v1/alumnos/1/documentos",  
    "curso": "/api/v1/cursos/5º Primaria"  
  }  
}
```

Pregunta

¿Qué ventaja tiene incluir enlaces en la respuesta? ¿Cómo ayuda al cliente a navegar por la API?

Respuesta razonada

Permite descubrir operaciones relacionadas desde la propia representación y reduce la necesidad de codificar rutas dispersas en el cliente.

Resultado esperado global Al final del ejercicio, el AlumnoController debe: Tener métodos cortos y declarativos.

- Usar @PathVariable, @RequestParam, @RequestBody, @RequestHeader.
- Devolver ResponseEntity con códigos de estado correctos.
- Incluir la cabecera Location en el POST.
- Tener la lógica extraída a métodos privados.
- Estar listo para que, en el siguiente punto, esa lógica se mueva a un servicio.
-

Conclusión y enlace al siguiente punto

En este punto 2.1 hemos profundizado en la capa de controlador: Qué es un controlador y cuál es su responsabilidad.

- @Controller vs @RestController.

Punto 2.2 - Capas de la aplicación: Servicio

Contexto del ejercicio: Vamos a extraer la lógica de negocio del AlumnoController a un AlumnoService. El controlador se quedará solo con el manejo HTTP. Añadiremos una excepción de negocio para el caso de DNI duplicado, y un manejador simple para convertirla en una respuesta 409. Requisitos previos: Tener el proyecto mi-proyecto con el AlumnoController y el AlumnoDTO del punto 2.1.

Nota 2026 - separación progresiva. El servicio se introduce sin adelantar todavía la capa de repositorio. La lista en memoria vive temporalmente en AlumnoService; la regla de DNI duplicado se expresa con NegocioException, el controlador la traduce a 409 Conflict y se mantiene la generación de ID segura heredada de M1. La separación DTO/entidad se explica aquí como patrón y se materializará cuando exista un modelo de persistencia distinto.

Paso 1 - Repasar el estado actual

Abre el AlumnoController y observa su contenido. Verás que tiene la lista de alumnos, la lógica de generación de ID, la búsqueda, la persistencia y el manejo HTTP, todo mezclado. Arranca la aplicación y prueba el CRUD completo para asegurarte de que funciona antes de refactorizar.

Pregunta

¿Qué partes del controlador son lógica de negocio y cuáles son manejo HTTP?

Respuesta razonada

Rutas, cabeceras, parámetros y ResponseEntity son HTTP. Búsquedas, reglas, generación de IDs y cambios de estado pertenecen al servicio/repositorio.

Paso 2 - Crear el paquete service

Crea un paquete service dentro del paquete raíz es.mecd.demo.miproyecto. Este paquete contendrá las clases de servicio. En el IDE, haz clic derecho sobre es.mecd.demo.miproyecto y selecciona New > Package. Escribe service.

Pregunta

¿Por qué conviene tener un paquete específico para los servicios?

Respuesta razonada

Hace visible la arquitectura, facilita navegación y evita mezclar lógica de aplicación con controladores o acceso a datos.

Paso 3 - Crear la excepción de negocio

Antes de crear el servicio, necesitamos una excepción para las violaciones de reglas de negocio. Crea el paquete exception y dentro la clase NegocioException:

```
package es.mecd.demo.miproyecto.exception;
public class NegocioException extends RuntimeException {
    public NegocioException(String mensaje) {
        super(mensaje);
    }
    public NegocioException(String mensaje, Throwable causa) {
        super(mensaje, causa);
    }
}
```

Esta clase extiende `RuntimeException`, lo que significa que no es una excepción “checked”: no obliga a los métodos que la lanzan a declararla con `throws`. Eso simplifica el código. Tiene dos constructores: uno con solo el mensaje y otro con mensaje y causa. El segundo es útil cuando la excepción de negocio envuelve otra excepción (por ejemplo, una de base de datos).

Pregunta

¿Qué diferencia hay entre extender `RuntimeException` y extender `Exception`?

Respuesta razonada

Una `RuntimeException` es no comprobada: no obliga a declarar o capturarla. Una excepción que extiende `Exception` sí es comprobada y forma parte explícita de la firma o del manejo.

Paso 4 - Crear el `AlumnoService`

Crea la clase `AlumnoService` en el paquete `service`:

```
package es.mecd.demo.miproyecto.service;
import es.mecd.demo.miproyecto.dto.AlumnoDTO;
import es.mecd.demo.miproyecto.exception.NegocioException;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
@Service
public class AlumnoService {
    private final List<AlumnoDTO> alumnos = new ArrayList<>() {
        add(new AlumnoDTO("1", "Ana", "García López", "12345678A",
            LocalDate.of(2010, 5, 12), "5º Primaria"),
            new AlumnoDTO("2", "Luis", "Pérez Ruiz", "87654321B",
            LocalDate.of(2009, 9, 3), "6º Primaria"));
    };
    public List<AlumnoDTO> listar() {
        return new ArrayList<>(alumnos);
    }
    public List<AlumnoDTO> listarPorCurso(String curso) {
        return alumnos.stream()
            .filter(a -> a.getCurso().equalsIgnoreCase(curso))
            .toList();
    }
    public Optional<AlumnoDTO> consultar(String id) {
        return alumnos.stream()
            .filter(a -> a.getIdentificador().equals(id))
            .findFirst();
    }
    public AlumnoDTO crear(AlumnoDTO dto) {
        if (existePorDni(dto.getDni())) {
            throw new NegocioException(
                "Ya existe un alumno con el DNI " + dto.getDni());
        }
        if (dto.getIdentificador() == null || dto.getIdentificador().isBlank()) {
            dto.setIdentificador(String.valueOf(alumnos.stream()
                .map(AlumnoDTO::getIdentificador)
                .filter(id -> id != null && id.matches("\\d+"))
                .mapToInt(Integer::parseInt)
                .max()));
        }
    }
}
```

```

        .orElse(0) + 1));
    }
    alumnos.add(dto);
    return dto;
}
public Optional<AlumnoDTO> actualizar(String id, AlumnoDTO dto) {
    return consultar(id).map(existente -> {
        dto.setIdentificador(id);
        alumnos.set(alumnos.indexOf(existente), dto);
        return dto;
    });
}
public Optional<AlumnoDTO> actualizarParcial(String id, java.util.Map<String, Object> cambios) {
    return consultar(id).map(alumno -> {
        if (cambios.containsKey("nombre")) {
            alumno.setNombre((String) cambios.get("nombre"));
        }
        if (cambios.containsKey("apellidos")) {
            alumno.setApellidos((String) cambios.get("apellidos"));
        }
        if (cambios.containsKey("curso")) {
            alumno.setCurso((String) cambios.get("curso"));
        }
        return alumno;
    });
}
public boolean eliminar(String id) {
    return alumnos.removeIf(a -> a.getIdentificador().equals(id));
}
private boolean existePorDni(String dni) {
    return alumnos.stream().anyMatch(a -> a.getDni().equals(dni));
}
}

```

Vamos a analizar los elementos clave de esta clase. La anotación `@Service` marca la clase como servicio. Spring la detectará al arrancar y la registrará como bean. La lista de alumnos es ahora un campo del servicio, no del controlador. El servicio es el dueño de los datos (en este caso, en memoria; en el Módulo 4 será el repositorio). Los métodos públicos (listar, consultar, crear, actualizar, actualizarParcial, eliminar) contienen la lógica de negocio. Cada uno hace una cosa y la hace bien. El método crear valida que el DNI no exista. Si existe, lanza `NegocioException`. Si no, asigna un ID y añade el alumno. Fíjate en que el servicio no construye una respuesta HTTP: solo devuelve el DTO o lanza una excepción. El método `existePorDni` es privado. Es una utilidad interna del servicio. No forma parte de la interfaz pública.

Pregunta

¿Por qué el método crear lanza una excepción en lugar de devolver un código de error?

Respuesta razonada

Porque el servicio debe expresar el conflicto del dominio sin decidir cómo se representa en HTTP; el controlador o un manejador traduce después esa excepción.

Paso 5 - Refactorizar el controlador para que use el servicio

Ahora vamos a modificar el `AlumnoController` para que delegue en el servicio. El controlador ya no tendrá la lista de alumnos ni la lógica de búsqueda: solo tendrá el manejo HTTP.

```
package es.mecd.demo.miproyecto.controller;
import es.mecd.demo.miproyecto.dto.AlumnoDTO;
import es.mecd.demo.miproyecto.exception.NegocioException;
import es.mecd.demo.miproyecto.service.AlumnoService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;
import java.util.Map;
@RestController
@RequestMapping("/api/v1/alumnos")
public class AlumnoController {
    private final AlumnoService service;
    public AlumnoController(AlumnoService service) {
        this.service = service;
    }
    @GetMapping
    public List<AlumnoDTO> listar(@RequestParam(required = false) String curso) {
        if (curso != null && !curso.isBlank()) {
            return service.listarPorCurso(curso);
        }
        return service.listar();
    }
    @GetMapping("/{id}")
    public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) {
        return service.consultar(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
    @PostMapping
    public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
        AlumnoDTO creado = service.crear(dto);
        URI location = URI.create("/api/v1/alumnos/" + creado.getIdentificador());
        return ResponseEntity.created(location).body(creado);
    }
    @PutMapping("/{id}")
    public ResponseEntity<AlumnoDTO> actualizar(
        @PathVariable String id,
        @RequestBody AlumnoDTO dto) {
        return service.actualizar(id, dto)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
    @PatchMapping("/{id}")
    public ResponseEntity<AlumnoDTO> actualizarParcial(
        @PathVariable String id,
        @RequestBody Map<String, Object> cambios) {
        return service.actualizarParcial(id, cambios)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable String id) {
        return service.eliminar(id)
            ? ResponseEntity.noContent().build()
            : ResponseEntity.notFound().build();
    }
}
```

```

    }
    @ExceptionHandler(NegocioException.class)
    public ResponseEntity<Map<String, Object>> handleNegocio(NegocioException ex) {
        Map<String, Object> error = Map.of(
            "status", 409,
            "error", "Conflict",
            "message", ex.getMessage()
        );
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }
}

```

Fíjate en los cambios: El controlador ya no tiene la lista de alumnos. Ahora tiene una dependencia del servicio.

- La inyección es por constructor. El campo service es final y se asigna en el constructor.
- Cada método delega en el servicio. El controlador no busca, no genera IDs, no persiste. Solo llama al servicio y construye la
- respuesta. El método handleNegocio captura la NegocioException y devuelve un 409 con un JSON de error. Es un manejador local del
- controlador; en el Módulo 5 lo haremos global.

Pregunta

¿Qué ventaja tiene que el controlador ya no sepa cómo se busca un alumno?

Respuesta razonada

Permite sustituir la estrategia interna de búsqueda o almacenamiento sin modificar el contrato web.

Paso 6 - Arrancar y probar el CRUD completo

Arranca la aplicación y prueba todos los endpoints:

Listar

```
curl http://localhost:8080/api/v1/alumnos
```

Consultar

```
curl http://localhost:8080/api/v1/alumnos/1
```

Crear

```

curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"María","apellidos":"López","dni":"33333333E","fechaNacimiento":"2011-03-20","curso":1}'

```

Actualizar

```

curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{"nombre":"Ana","apellidos":"García López","dni":"12345678A","fechaNacimiento":"2010-05-12","curso":1}'

```

Actualizar parcial

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{"curso": "6º Primaria B"}'
```

Eliminar

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/2
```

Todo debe funcionar igual que antes. La refactorización no debe romper nada.

Pregunta

¿Qué ventaja tiene que el controlador sea ahora más corto?

Respuesta razonada

Un controlador corto concentra la frontera HTTP, reduce ramas y dependencias y resulta más sencillo de probar y mantener.

Paso 7 - Probar el error de negocio

Ahora vamos a probar la regla de negocio. Intenta crear un alumno con un DNI que ya existe:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Otro","apellidos":"Alumno","dni":"12345678A","fechaNacimiento":"2010-01-01","curso":'}
```

Verás algo como:

```
HTTP/1.1 409
Content-Type: application/json
{"status":409,"error":"Conflict","message":"Ya existe un alumno con el DNI 12345678A"}
```

El servicio ha detectado el duplicado y ha lanzado `NegocioException`. El manejador del controlador la ha capturado y ha devuelto un 409 con un JSON descriptivo.

Pregunta

¿Qué diferencia hay entre este 409 y un 400? ¿Por qué es 409 y no 400?

Respuesta razonada

400 indica que la petición no puede interpretarse o es inválida como mensaje; 409 indica que el mensaje es válido pero entra en conflicto con el estado o una regla del dominio.

Paso 8 - Probar un error técnico

Ahora vamos a provocar un error técnico para ver la diferencia. Envía un POST con un JSON mal formado:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro", "apellidos":"Sánchez",}'
```

Verás un 400 Bad Request, no un 409. Ese error lo genera Spring Boot automáticamente al no poder deserializar el JSON. Es un error del cliente, no una violación de regla de negocio.

Pregunta

¿Qué diferencia hay entre un 400 y un 409 en cuanto a quién tiene la culpa?

Respuesta razonada

Un 400 se detecta en la frontera de entrada por formato o datos no interpretables; un 409 aparece después de interpretar correctamente la petición y aplicar una regla de negocio.

Paso 9 - Analizar la separación de capas

Ahora que tenemos controlador y servicio, vamos a analizar la separación:

Responsabilidad	Controlador	Servicio
Manejo HTTP	Sí	No
Extracción de parámetros	Sí	No
Construcción de respuestas	Sí	No
Códigos de estado	Sí	No
Lógica de negocio	No	Sí
Validaciones de negocio	No	Sí
Generación de IDs	No	Sí
Persistencia (en memoria, en este punto)	No	Sí
Excepciones de negocio	No	Sí

El controlador solo sabe de HTTP. El servicio solo sabe de negocio. Cada uno tiene su responsabilidad.

Pregunta

¿Qué ventaja tiene esta separación cuando hay que cambiar una regla de negocio?

Respuesta razonada

La regla queda concentrada en el servicio y puede cambiar sin modificar rutas, cabeceras ni serialización del controlador.

Paso 10 - Errores comunes del ejercicio

Error Causa Solución

NoSuchBeanDefinitionException:

Falta @Service Añadir la anotación

AlumnoService

Verificar la inyección por NullPointerException en el controlador El servicio es null constructor El 409 no aparece Falta @ExceptionHandler Añadirlo al controlador El error de negocio devuelve 500 No se captura la excepción Añadir el manejador La lógica sigue en el controlador No se ha movido al servicio Moverla Se ha El servicio tiene lógica HTTP Quitarlo importado ResponseEntity La lista es inmutable Usar new ArrayList<>(...)

UnsupportedOperationException

Paso 11 - Reto resuelto - Servicio para el recurso Expediente

Reto: Crear un ExpedienteService aplicando el mismo patrón que hemos usado con AlumnoService. El

controlador ExpedienteController debe delegar en el servicio.

Solución paso a paso:**Paso 1: Crear el ExpedienteService:**

```

package es.mecd.demo.miproyecto.service;
import es.mecd.demo.miproyecto.dto.ExpedienteDTO;
import es.mecd.demo.miproyecto.exception.NegocioException;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
@Service
public class ExpedienteService {
    private final List<ExpedienteDTO> expedientes = new ArrayList<> (List.of(
        new ExpedienteDTO("1", "Ana García", "12345678A",
            "EN_TRAMITE", "BECA", LocalDate.of(2025, 1, 15), 1500.0),
        new ExpedienteDTO("2", "Luis Pérez", "87654321B",
            "RESUELTA", "AYUDA_LIBROS", LocalDate.of(2025, 1, 10), 300.0)
    ));
    public List<ExpedienteDTO> listar() {
        return new ArrayList<>(expedientes);
    }
    public Optional<ExpedienteDTO> consultar(String id) {
        return expedientes.stream()
            .filter(e -> e.getIdentificador().equals(id))
            .findFirst();
    }
    public ExpedienteDTO crear(ExpedienteDTO dto) {
        if (dto.getIdentificador() == null || dto.getIdentificador().isBlank()) {
            dto.setIdentificador(String.valueOf(expedientes.size() + 1));
        }
        expedientes.add(dto);
        return dto;
    }
    public Optional<ExpedienteDTO> actualizar(String id, ExpedienteDTO dto) {
        return consultar(id).map(existente -> {
            dto.setIdentificador(id);
            expedientes.set(expedientes.indexOf(existente), dto);
            return dto;
        });
    }
    public boolean eliminar(String id) {
        return expedientes.removeIf(e -> e.getIdentificador().equals(id));
    }
}

```

Paso 2: Refactorizar el ExpedienteController para que use el servicio:

```

@RestController
@RequestMapping("/api/v1/expedientes")
public class ExpedienteController {
    private final ExpedienteService service;
    public ExpedienteController(ExpedienteService service) {
        this.service = service;
    }
    @GetMapping
    public List<ExpedienteDTO> listar() {
        return service.listar();
    }
}

```

```

    }
    @GetMapping("/{id}")
    public ResponseEntity<ExpedienteDTO> consultar(@PathVariable String id) {
        return service.consultar(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
    @PostMapping
    public ResponseEntity<ExpedienteDTO> crear(@RequestBody ExpedienteDTO dto) {
        ExpedienteDTO creado = service.crear(dto);
        URI location = URI.create("/api/v1/expedientes/" + creado.getIdentificador());
        return ResponseEntity.created(location).body(creado);
    }
    @PutMapping("/{id}")
    public ResponseEntity<ExpedienteDTO> actualizar(
        @PathVariable String id,
        @RequestBody ExpedienteDTO dto) {
        return service.actualizar(id, dto)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable String id) {
        return service.eliminar(id)
            ? ResponseEntity.noContent().build()
            : ResponseEntity.notFound().build();
    }
}

```

Paso 3: Arrancar y probar. Los endpoints de expedientes deben seguir funcionando igual, pero ahora la lógica está en el servicio.

Pregunta

¿Qué ventaja tiene tener la lógica de expedientes en un servicio separado?

Respuesta razonada

Mantiene cohesión por recurso y evita que reglas de dominios distintos queden mezcladas en una misma clase.

Resultado esperado global Al final del ejercicio, deberías tener: Un AlumnoService con la lógica de negocio.

- Un AlumnoController que solo maneja HTTP y delega en el servicio.
- Una NegocioException para violaciones de reglas.
- Un @ExceptionHandler en el controlador que convierte la excepción en un 409.
- (Opcional) Un ExpedienteService con el mismo patrón.
- Y la estructura del proyecto debe incluir:

```

src/main/java/es/mecd/demo/miproyecto/
├── controller/
│   ├── AlumnoController.java
│   └── ExpedienteController.java
├── service/
│   ├── AlumnoService.java
│   └── ExpedienteService.java

```

```
├── dto/
│   ├── AlumnoDTO.java
│   └── ExpedienteDTO.java
└── exception/
    └── NegocioException.java
```

Conclusión y enlace al siguiente punto

En este punto 2.2 hemos profundizado en la capa de servicio: Qué es un servicio y cuál es su responsabilidad.

- @Service frente a @Component, @Repository y @Controller.
- Cómo Spring detecta y registra los servicios.

Punto 2.3 - Capas de la aplicación: Repositorio

Contexto del ejercicio

Partimos del estado cerrado de 2.2. Los controladores ya delegan en servicios, pero AlumnoService y ExpedienteService todavía almacenan datos en colecciones propias. En este punto extraeremos esa responsabilidad a repositorios en memoria.

El objetivo no es simplemente crear dos clases nuevas. Queremos poder demostrar que:

Controller -> Service -> Repository

funciona sin cambiar el contrato HTTP acumulado desde M0 y M1.

Requisitos previos

Antes de modificar nada deben seguir funcionando:

```
./mvnw test
./mvnw -DskipTests package
```

Y, con la aplicación arrancada:

```
curl -i http://localhost:8080/hola
curl -i http://localhost:8080/adios
curl -i http://localhost:8080/api/v1/alumnos
curl -i http://localhost:8080/api/v1/expedientes/ejemplo
```

Paso 1 - Repasar el estado actual

Abre AlumnoService.java y localiza las dos responsabilidades que vamos a separar.

Responsabilidad de negocio:

```
if (existePorDni(dto.getDni())) {
    throw new NegocioException(...);
}
```

Responsabilidad de almacenamiento:

```
private final List<AlumnoDTO> alumnos = ...;
```

También encontrarás búsquedas, inserciones y borrados directamente sobre esa colección.

Haz la misma inspección en ExpedienteService.

Pregunta

¿Cuál de estas dos frases pertenece al negocio y cuál al acceso a datos?

"no puede haber dos alumnos con el mismo DNI"

"buscar un alumno cuyo ID sea 3"

Respuesta razonada

La primera expresa una regla de negocio y debe permanecer en el servicio. La segunda es una operación de acceso a datos y debe delegarse al repositorio.

Paso 2 - Crear el paquete repository

Crea:

M2/proyecto/src/main/java/es/mecd/demo/miproyecto/repository/

La estructura quedará progresivamente así:

```
es.mecd.demo.miproyecto
├── controller
├── dto
├── exception
├── repository
└── service
```

Desde IntelliJ IDEA

Clic derecho en es.mecd.demo.miproyecto → **New > Package** → repository.

Desde Eclipse

Clic derecho en src/main/java o en el paquete raíz → **New > Package** → escribe el nombre completo.

Desde VS Code

Crea la carpeta repository dentro del paquete raíz respetando exactamente el árbol Java.

Desde consola

En sistemas tipo Unix:

```
mkdir -p src/main/java/es/mecd/demo/miproyecto/repository
```

En PowerShell:

```
New-Item -ItemType Directory -Force `
  src/main/java/es/mecd/demo/miproyecto/repository
```

Paso 3 - Crear AlumnoRepository

Crea:

src/main/java/es/mecd/demo/miproyecto/repository/AlumnoRepository.java

Usa esta implementación:

```
package es.mecd.demo.miproyecto.repository;

import es.mecd.demo.miproyecto.dto.AlumnoDTO;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

@Repository
public class AlumnoRepository {

    private final Map<String, AlumnoDTO> almacen = new ConcurrentHashMap<>();
    private final AtomicInteger secuencia = new AtomicInteger(0);

    public AlumnoRepository() {
        guardarInicial(new AlumnoDTO(
            "1", "Ana", "García López", "DNI-DEMO-01",
            LocalDate.of(2010, 5, 12), "5º Primaria"));
        guardarInicial(new AlumnoDTO(
            "2", "Luis", "Pérez Ruiz", "DNI-DEMO-02",
            LocalDate.of(2009, 9, 3), "6º Primaria"));
    }

    public Optional<AlumnoDTO> buscarPorId(String id) {
        return Optional.ofNullable(almacen.get(id)).map(this::copiar);
    }

    public List<AlumnoDTO> listarTodos() {
        return almacen.values().stream()
            .map(this::copiar)
            .sorted(Comparator.comparingInt(this::idNumerico))
            .toList();
    }

    public Optional<AlumnoDTO> buscarPorDni(String dni) {
        return almacen.values().stream()
            .filter(a -> Objects.equals(a.getDni(), dni))
            .findFirst()
            .map(this::copiar);
    }

    public boolean existePorDni(String dni) {
        return almacen.values().stream()
            .anyMatch(a -> Objects.equals(a.getDni(), dni));
    }

    public AlumnoDTO guardar(AlumnoDTO alumno) {
        AlumnoDTO copia = copiar(alumno);
        almacen.put(copia.getIdentificador(), copia);
        actualizarSecuencia(copia.getIdentificador());
        return copiar(copia);
    }
}
```

```

    }

    public boolean eliminar(String id) {
        return almacen.remove(id) != null;
    }

    public int contar() {
        return almacen.size();
    }

    public String siguienteIdentificador() {
        return String.valueOf(secuencia.incrementAndGet());
    }

    private void guardarInicial(AlumnoDTO alumno) {
        almacen.put(alumno.getIdentificador(), copiar(alumno));
        actualizarSecuencia(alumno.getIdentificador());
    }

    private void actualizarSecuencia(String id) {
        if (id != null && id.matches("\\d+")) {
            secuencia.accumulateAndGet(Integer.parseInt(id), Math::max);
        }
    }

    private int idNumerico(AlumnoDTO alumno) {
        String id = alumno.getIdentificador();
        return id != null && id.matches("\\d+")
            ? Integer.parseInt(id)
            : Integer.MAX_VALUE;
    }

    private AlumnoDTO copiar(AlumnoDTO original) {
        AlumnoDTO copia = new AlumnoDTO(
            original.getIdentificador(),
            original.getNombre(),
            original.getApellidos(),
            original.getDni(),
            original.getFechaNacimiento(),
            original.getCurso());
        copia.setDocumentos(original.getDocumentos() == null
            ? null
            : List.copyOf(original.getDocumentos()));
        return copia;
    }
}

```

Qué debes entender del código

ConcurrentHashMap protege la estructura ante accesos concurrentes. Optional expresa la posible ausencia. listarTodos() no expone el mapa. copiar() evita que un objeto obtenido desde fuera pueda modificar directamente el almacenado.

La fuente original utilizaba contar() + 1 para generar IDs. Conservamos contar() porque forma parte de la API pedagógica del repositorio, pero **no lo usamos como generador**, ya que después de borrar un elemento podría repetir un ID. AtomicInteger mantiene una secuencia monotónica para este laboratorio.

Ejecuta:

```
./mvnw test
```

Paso 4 - Refactorizar AlumnoService

Sustituye la colección interna por una dependencia del repositorio:

```
private final AlumnoRepository repositorio;

public AlumnoService(AlumnoRepository repositorio) {
    this.repositorio = repositorio;
}
```

El listado acumulativo de 2.1 y 2.2 mantiene filtro, orden y paginación:

```
public List<AlumnoDTO> listar(
    String curso,
    String sort,
    int page,
    int size) {

    var stream = repositorio.listarTodos().stream();

    if (curso != null && !curso.isBlank()) {
        stream = stream.filter(
            a -> a.getCurso().equalsIgnoreCase(curso));
    }

    if ("nombre".equalsIgnoreCase(sort)) {
        stream = stream.sorted(
            (a, b) -> a.getNombre().compareToIgnoreCase(b.getNombre()));
    } else if ("apellidos".equalsIgnoreCase(sort)) {
        stream = stream.sorted(
            (a, b) -> a.getApellidos().compareToIgnoreCase(b.getApellidos()));
    }

    return stream
        .skip((long) page * size)
        .limit(size)
        .toList();
}
```

La consulta:

```
public Optional<AlumnoDTO> consultar(String id) {
    return repositorio.buscarPorId(id);
}
```

La creación conserva la regla de negocio:

```
public AlumnoDTO crear(AlumnoDTO dto) {
    if (repositorio.existePorDni(dto.getDni())) {
        throw new NegocioException(
            "Ya existe un alumno con el DNI " + dto.getDni());
    }

    if (dto.getIdentificador() == null
        || dto.getIdentificador().isBlank()) {
        dto.setIdentificador(repositorio.siguienteIdentificador());
    }
}
```

```

    return repositorio.guardar(dto);
}

```

PUT:

```

public Optional<AlumnoDTO> actualizar(String id, AlumnoDTO dto) {
    return repositorio.buscarPorId(id).map(existente -> {
        dto.setIdentificador(id);
        return repositorio.guardar(dto);
    });
}

```

PATCH mantiene también el campo dni heredado de M1:

```

public Optional<AlumnoDTO> actualizarParcial(
    String id,
    Map<String, Object> cambios) {

    return repositorio.buscarPorId(id).map(alumno -> {
        if (cambios.containsKey("nombre")) {
            alumno.setNombre((String) cambios.get("nombre"));
        }
        if (cambios.containsKey("apellidos")) {
            alumno.setApellidos((String) cambios.get("apellidos"));
        }
        if (cambios.containsKey("dni")) {
            alumno.setDni((String) cambios.get("dni"));
        }
        if (cambios.containsKey("curso")) {
            alumno.setCurso((String) cambios.get("curso"));
        }
        return repositorio.guardar(alumno);
    });
}

```

Eliminar:

```

public boolean eliminar(String id) {
    return repositorio.eliminar(id);
}

```

Promocionar:

```

public void promocionar() {
    repositorio.listarTodos().forEach(alumno -> {
        alumno.setCurso(alumno.getCurso() + " (promocionado)");
        repositorio.guardar(alumno);
    });
}

```

Pregunta

¿Por qué existePorDni es una operación del repositorio pero decidir que un DNI repetido invalida la creación sigue siendo responsabilidad del servicio?

Respuesta razonada

El repositorio informa sobre el estado de los datos. El servicio convierte esa información en una regla de negocio.

Paso 5 - Arrancar y probar el CRUD completo

Ejecuta:

```
./mvnw test
./mvnw -DskipTests package
./mvnw spring-boot:run
```

En otra terminal:

Listar

```
curl -i http://localhost:8080/api/v1/alumnos
```

Consultar

```
curl -i http://localhost:8080/api/v1/alumnos/1
```

Crear

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"María","apellidos":"López","dni":"DNI-DEMO-03","fechaNacimiento":"2011-03-20","curso":"6º Primaria B"}'
```

Actualizar

```
curl -i -X PUT http://localhost:8080/api/v1/alumnos/3 \
-H "Content-Type: application/json" \
-d '{"nombre":"María","apellidos":"López","dni":"DNI-DEMO-03","fechaNacimiento":"2011-03-20","curso":"6º Primaria B"}'
```

Actualización parcial

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/3 \
-H "Content-Type: application/json" \
-d '{"curso":"6º Primaria B"}'
```

DNI duplicado

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Otra Ana","apellidos":"Duplicada","dni":"DNI-DEMO-01","fechaNacimiento":"2010-01-01","curso":"6º Primaria B"}'
```

Eliminar

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/3
```

Comprueba que siguen apareciendo los mismos contratos externos: 200, 201 + Location, 204, 404 y 409 cuando corresponde.

Prueba importante de generación de ID

Para comprobar que no hemos reintroducido `contar()+1`:

1. crea el alumno 3;
2. crea el alumno 4;
3. elimina el alumno 3;
4. crea otro alumno.

El nuevo identificador debe ser 5, no 4.

Paso 6 - Analizar la separación de capas

Completa esta matriz y compruébala contra el código:

Responsabilidad	Controller	Service	Repository
rutasy métodos HTTP	sí	no	no
extracción de parámetros	sí	no	no
ResponseEntity y status	sí	no	no
reglas de negocio	no	sí	no
filtro, orden y paginación	no	sí	no
regla de DNI duplicado	no	sí	no
obtener datos por ID/DNI	no	solicita	sí
guardar/eliminar	no	solicita	sí
conocer ConcurrentHashMap	no	no	sí
proteger el estado interno	no	no	sí

Busca fugas de capas:

```
grep -R "ResponseEntity\|HttpStatus\|RequestParam\|PathVariable" \
src/main/java/es/meecd/demo/miproyecto/repository || true
```

La búsqueda no debería encontrar dependencias HTTP dentro del repositorio.

Paso 7 - Probar el aislamiento del repositorio

La fuente denomina este paso “probar la inmutabilidad”. Lo que realmente comprobaremos es que el estado interno no se puede modificar accidentalmente desde fuera.

Añade temporalmente una prueba o ejecútala posteriormente como test automatizado:

```
AlumnoRepository repo = new AlumnoRepository();
```

```
var lista = repo.listarTodos();
lista.clear();
```

```
assertEquals(2, repo.listarTodos().size());
```

Ahora comprueba también el objeto:

```
AlumnoDTO obtenido = repo.buscarPorId("1").orElseThrow();
obtenido.setNombre("CAMBIO EXTERNO");
```

```
AlumnoDTO almacenado = repo.buscarPorId("1").orElseThrow();
assertEquals("Ana", almacenado.getNombre());
```

Esta segunda comprobación es más fuerte que la de la fuente original, porque detecta las fugas de referencias mutables.

Pregunta

¿Por qué una lista nueva no garantiza por sí sola que el contenido interno esté protegido?

Respuesta razonada

Porque puede contener los mismos objetos mutables que el repositorio. Necesitamos decidir también si debemos copiar esos elementos.

Paso 8 - Probar accesos concurrentes

Con la aplicación arrancada, crea un pequeño script que envíe varias creaciones en paralelo.

En Bash:

```
for n in 10 11 12 13 14; do
  curl -s -X POST http://localhost:8080/api/v1/alumnos \
    -H "Content-Type: application/json" \
    -d "{\"nombre\":\"Alumno$n\",\"apellidos\":\"Paralelo\",\"dni\":\"DNI-PAR-$n\",\"fechaNacimiento\":\"2011-01-01\",\"curso\":\"5º Primaria\"}"
done
wait
```

```
curl -s http://localhost:8080/api/v1/alumnos
```

En PowerShell puedes utilizar trabajos:

```
10..14 | ForEach-Object {
    $n = $_
    Start-Job -ScriptBlock {
        param($n)
        $body = @{
            nombre = "Alumno$n"
            apellidos = "Paralelo"
            dni = "DNI-PAR-$n"
            fechaNacimiento = "2011-01-01"
            curso = "5º Primaria"
        } | ConvertTo-Json
        Invoke-RestMethod -Method Post `
            -Uri http://localhost:8080/api/v1/alumnos `
            -ContentType application/json `
            -Body $body
    } -ArgumentList $n
} | Wait-Job | Receive-Job
```

Comprueba que las creaciones válidas reciben identificadores distintos y que el repositorio sigue respondiendo con una colección coherente.

Qué demuestra y qué no demuestra

Demuestra que nuestra implementación soporta este nivel de acceso concurrente sin depender de una colección no segura. **No demuestra** propiedades transaccionales equivalentes a una base de datos.

Paso 9 - Errores comunes del ejercicio

Síntoma	Causa	Comprobación / solución
NoSuchBeanDefinitionException para el repositorio	falta @Repository o paquete mal ubicado	revisar paquete y anotación
el controlador cambia para conocer el repositorio	se ha saltado la capa de servicio	controller debe seguir usando service
el servicio conserva List<AlumnoDTO> alumnos	refactor incompleto	mover estado al repository
el repositorio contiene ResponseEntity	HTTP se filtró a datos	eliminar dependencia web
buscarPorId devuelve null	contrato débil	usar Optional
listarTodos devuelve el mapa/lista interna	encapsulamiento roto	devolver copias

Síntoma	Causa	Comprobación / solución
un DTO consultado modifica el almacenado	copia superficial	copiar elementos mutables
aparecen IDs repetidos tras DELETE	contar()+1	usar secuencia monotonica
el DNI duplicado se valida en repository	lógica de negocio desplazada	mantener regla en service
se pierde sort, page, size o PATCH de DNI	se copió literalmente el ejemplo simplificado	conservar contrato acumulativo

Después de cualquier corrección ejecuta:

```
./mvnw test
./mvnw -DskipTests package
```

Paso 10 - Reto resuelto: repositorio para Expediente

La fuente termina 2.3 aplicando el mismo patrón a Expediente. Vamos a completarlo sin perder /ejemplo ni /eco de M1.

10.1 Crear ExpedienteRepository

Crea:

src/main/java/es/mecd/demo/miproyecto/repository/ExpedienteRepository.java

Implementación:

```
package es.mecd.demo.miproyecto.repository;

import es.mecd.demo.miproyecto.dto.ExpedienteDTO;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

@Repository
public class ExpedienteRepository {

    private final Map<String, ExpedienteDTO> almacen =
        new ConcurrentHashMap<>();
    private final AtomicInteger secuencia = new AtomicInteger(0);

    public ExpedienteRepository() {
        guardarInicial(new ExpedienteDTO(
            "1", "Ana García", "12345678A", "EN_TRAMITE", "BECA",
            LocalDate.of(2025, 1, 15), 1500.0, true,
            List.of("DNI.pdf")));
        guardarInicial(new ExpedienteDTO(
            "2", "Luis Pérez", "87654321B", "RESUELTA", "AYUDA_LIBROS",
            LocalDate.of(2025, 1, 10), 300.0, true,
```

```

        List.of("Solicitud.pdf"))));
    }

    public Optional<ExpedienteDTO> buscarPorId(String id) {
        return Optional.ofNullable(almacen.get(id)).map(this::copiar);
    }

    public List<ExpedienteDTO> listarTodos() {
        return almacen.values().stream()
            .map(this::copiar)
            .sorted(Comparator.comparingInt(this::idNumerico))
            .toList();
    }

    public ExpedienteDTO guardar(ExpedienteDTO expediente) {
        ExpedienteDTO copia = copiar(expediente);
        almacen.put(copia.getIdentificador(), copia);
        actualizarSecuencia(copia.getIdentificador());
        return copiar(copia);
    }

    public boolean eliminar(String id) {
        return almacen.remove(id) != null;
    }

    public int contar() {
        return almacen.size();
    }

    public String siguienteIdentificador() {
        return String.valueOf(sequencia.incrementAndGet());
    }

    // guardarInicial, actualizarSecuencia, idNumerico y copiar
    // siguen el mismo patrón explicado en AlumnoRepository.
}

```

En el proyecto final esos helpers deben estar implementados completamente; no dejes comentarios como sustituto de código ejecutable.

10.2 Refactorizar ExpedienteService

Añade la dependencia:

```

private final ExpedienteRepository repositorio;

public ExpedienteService(ExpedienteRepository repositorio) {
    this.repositorio = repositorio;
}

```

Y delega:

```

public List<ExpedienteDTO> listar() {
    return repositorio.listarTodos();
}

public Optional<ExpedienteDTO> consultar(String id) {
    return repositorio.buscarPorId(id);
}

```

```

public ExpedienteDTO crear(ExpedienteDTO dto) {
    if (dto.getIdentificador() == null || dto.getIdentificador().isBlank()) {
        dto.setIdentificador(repositorio.siguienteIdentificador());
    }
    return repositorio.guardar(dto);
}

public Optional<ExpedienteDTO> actualizar(String id, ExpedienteDTO dto) {
    return repositorio.buscarPorId(id).map(existente -> {
        dto.setIdentificador(id);
        return repositorio.guardar(dto);
    });
}

public boolean eliminar(String id) {
    return repositorio.eliminar(id);
}

```

Los métodos pedagógicos heredados se conservan:

```

public ExpedienteDTO ejemplo() { ... }
public ExpedienteDTO eco(ExpedienteDTO dto) { ... }

```

10.3 Regresión del recurso

Arranca y comprueba:

```

curl -i http://localhost:8080/api/v1/expedientes/ejemplo
curl -i http://localhost:8080/api/v1/expedientes
curl -i http://localhost:8080/api/v1/expedientes/1

```

```

curl -i -X POST http://localhost:8080/api/v1/expedientes/eco \
-H "Content-Type: application/json" \
-d '{"titular":"Eco","dni":"11111111A","estado":"NUEVA","tipo":"BECA","fechaSolicitud":"2025-01-1"}'

```

Y prueba también POST/PUT/DELETE del CRUD que ya introdujimos en 2.2.

Pregunta final

¿Qué información de ConcurrentHashMap necesita conocer ExpedienteController?

Respuesta razonada

Ninguna. El controlador sólo conoce ExpedienteService. Esa es precisamente la separación que queremos conseguir.

Comprobación final de 2.3

Ejecuta:

```

./mvnw test
./mvnw -DskipTests package

```

La aplicación debe conservar todo lo aprendido antes y añadir una frontera de datos claramente separada. El resultado acumulativo es:

HTTP -> Controller -> Service -> Repository -> memoria

En 2.4 usaremos esta arquitectura ya separada para profundizar en inyección de dependencias y configuración de Spring.

Punto 2.4 - Inyección de dependencias y configuración

Contexto del ejercicio

Partimos del estado final de 2.3:

Controller -> Service -> Repository

Los beans ya se conectan por constructor, pero los repositorios todavía cargan datos de ejemplo por sí mismos. En este punto moveremos esa inicialización a configuración, añadiremos propiedades externas, perfiles y un bean personalizado.

Paso 1 - Repasar el estado actual

Comprueba la cadena de constructores:

```
public AlumnoController(AlumnoService service) { ... }  
public AlumnoService(AlumnoRepository repositorio) { ... }
```

Ninguna capa debe construir manualmente la siguiente con new.

Ejecuta:

```
./mvnw test  
./mvnw -DskipTests package
```

Pregunta

¿Por qué el constructor de AlumnoRepository sigue teniendo una responsabilidad que no pertenece al acceso a datos?

Respuesta razonada

Porque decide qué datos iniciales existen. El repositorio debe almacenar y recuperar; la decisión de precargar datos de demostración puede externalizarse a configuración de arranque.

Paso 2 - Crear el paquete config

Crea:

```
src/main/java/es/me.cd/demo/miproyecto/config/
```

Desde IntelliJ IDEA: **New > Package**. Desde Eclipse: **New > Package**. Desde VS Code o consola crea la carpeta correspondiente al paquete Java.

En Bash:

```
mkdir -p src/main/java/es/me.cd/demo/miproyecto/config
```

Paso 3 - Crear DatosInicialesConfig

Crea:

config/DatosInicialesConfig.java

Contenido:

```
package es.mecd.demo.miproyecto.config;

import es.mecd.demo.miproyecto.dto.AlumnoDTO;
import es.mecd.demo.miproyecto.dto.ExpedienteDTO;
import es.mecd.demo.miproyecto.repository.AlumnoRepository;
import es.mecd.demo.miproyecto.repository.ExpedienteRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.LocalDate;
import java.util.List;

@Configuration
public class DatosInicialesConfig {

    @Bean
    CommandLineRunner cargarDatos(
        AlumnoRepository alumnos,
        ExpedienteRepository expedientes) {

        return args -> {
            if (alumnos.contar() == 0) {
                alumnos.guardar(new AlumnoDTO(
                    "1", "Ana", "García López", "DNI-DEMO-01",
                    LocalDate.of(2010, 5, 12), "5º Primaria"));
                alumnos.guardar(new AlumnoDTO(
                    "2", "Luis", "Pérez Ruiz", "DNI-DEMO-02",
                    LocalDate.of(2009, 9, 3), "6º Primaria"));
            }

            if (expedientes.contar() == 0) {
                expedientes.guardar(new ExpedienteDTO(
                    "1", "Ana García", "12345678A",
                    "EN_TRAMITE", "BECA",
                    LocalDate.of(2025, 1, 15),
                    1500.0, true, List.of("DNI.pdf")));
                expedientes.guardar(new ExpedienteDTO(
                    "2", "Luis Pérez", "87654321B",
                    "RESUELTA", "AYUDA_LIBROS",
                    LocalDate.of(2025, 1, 10),
                    300.0, true, List.of("Solicitud.pdf")));
            }
        };
    }
}
```

Observa que Spring entrega los repositorios como parámetros. No construimos instancias nuevas.

Pregunta

¿Qué error conceptual cometeríamos escribiendo `new AlumnoRepository()` dentro del runner?

Respuesta razonada

Crearíamos una instancia distinta de la gestionada por Spring. El servicio y el runner podrían trabajar sobre repositorios diferentes.

Paso 4 - Limpiar los constructores de los repositorios

Elimina de AlumnoRepository y ExpedienteRepository la carga de datos iniciales.

Los campos permanecen:

```
private final Map<String, AlumnoDTO> almacen =  
    new ConcurrentHashMap<>();  
private final AtomicInteger secuencia = new AtomicInteger(0);
```

pero el constructor ya no debe decidir qué alumnos existen.

Puedes eliminar el constructor explícito si queda vacío.

Arranca la aplicación y comprueba:

```
curl -s http://localhost:8080/api/v1/alumnos  
curl -s http://localhost:8080/api/v1/expedientes
```

Deben seguir apareciendo dos elementos iniciales en cada colección.

Comprobación importante

Reinicia varias veces. Cada arranque empieza con un nuevo mapa en memoria y el runner carga exactamente una vez el conjunto inicial. Dentro de una misma ejecución no deben duplicarse los datos.

Paso 5 - Añadir propiedades personalizadas

Abre:

src/main/resources/application.properties

Mantén la propiedad heredada:

spring.application.name=mi-proyecto

Y añade:

```
app.nombre=Curso Spring Boot 2026  
app.version=2.4  
app.entorno=base  
app.descripcion=Aplicación pedagógica de arquitectura en capas
```

Usamos el prefijo app. para agrupar propiedades propias y reducir colisiones con propiedades de Spring Boot.

Paso 6 - Leer propiedades con @Value

Crea un controlador pequeño de configuración:

controller/InfoAplicacionController.java

Contenido:

```

package es.mecd.demo.miproyecto.controller;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.LinkedHashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/v1/info")
public class InfoAplicacionController {

    private final String nombre;
    private final String version;
    private final String entorno;
    private final String descripcion;

    public InfoAplicacionController(
        @Value("${app.nombre}") String nombre,
        @Value("${app.version}") String version,
        @Value("${app.entorno:desconocido}") String entorno,
        @Value("${app.descripcion:Sin descripción}") String descripcion) {
        this.nombre = nombre;
        this.version = version;
        this.entorno = entorno;
        this.descripcion = descripcion;
    }

    @GetMapping
    public Map<String, String> info() {
        Map<String, String> respuesta = new LinkedHashMap<>();
        respuesta.put("nombre", nombre);
        respuesta.put("version", version);
        respuesta.put("entorno", entorno);
        respuesta.put("descripcion", descripcion);
        return respuesta;
    }
}

```

La fuente original muestra @Value directamente sobre campos. La edición nueva lo aplica en el constructor para mantener visibles las dependencias de configuración y conservar campos final.

Pregunta

¿Qué ocurre con \${app.entorno:desconocido} si la propiedad no existe?

Respuesta razonada

Spring utiliza desconocido como valor por defecto. Sin :valor, una propiedad requerida ausente puede impedir la creación del bean.

Paso 7 - Exponer propiedades no sensibles en un endpoint

Arranca:

```
./mvnw spring-boot:run
```

Prueba:

```
curl -i http://localhost:8080/api/v1/info
```

Respuesta esperada equivalente a:

```
{
  "nombre": "Curso Spring Boot 2026",
  "version": "2.4",
  "entorno": "base",
  "descripcion": "Aplicación pedagógica de arquitectura en capas"
}
```

No añadas contraseñas, tokens ni secretos a este endpoint.

Paso 8 - Crear el perfil de desarrollo

Crea:

```
src/main/resources/application-dev.properties
```

Contenido:

```
app.entorno=desarrollo
app.descripcion=Entorno local de desarrollo
logging.level.es.mecd.demo.miproyecto=DEBUG
```

Arranca:

```
./mvnw spring-boot:run \
  -Dspring-boot.run.profiles=dev
```

Consulta:

```
curl -s http://localhost:8080/api/v1/info
```

entorno debe ser desarrollo, mientras las propiedades no redefinidas siguen procediendo de application.properties.

Paso 9 - Crear el perfil de producción

Crea:

```
src/main/resources/application-prod.properties
```

Contenido:

```
app.entorno=produccion
app.descripcion=Configuración de producción de ejemplo
logging.level.es.mecd.demo.miproyecto=INFO
```

Empaqueta:

```
./mvnw -DskipTests package
```

Arranca:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar \
  --spring.profiles.active=prod
```

Comprueba:

```
curl -s http://localhost:8080/api/v1/info
```

El endpoint debe reflejar el perfil activo sin haber recompilado el código.

Paso 10 - Probar CommandLineRunner

Para observar la inicialización sin acoplar el ejercicio a mensajes de log internos, comprueba el efecto:

1. inicia la aplicación;
2. consulta `/api/v1/alumnos`;
3. deben existir exactamente los dos alumnos iniciales;
4. consulta `/api/v1/expedientes`;
5. deben existir exactamente dos expedientes;
6. crea nuevos elementos;
7. dentro de esa ejecución deben persistir en memoria;
8. reinicia y volverás al conjunto inicial.

El último punto recuerda una limitación deliberada: aún no tenemos persistencia real.

Pregunta

¿Por qué el runner se ejecuta cuando ya puede utilizar los repositorios como beans?

Respuesta razonada

Porque Spring construye primero el contexto y resuelve las dependencias necesarias; después ejecuta los `CommandLineRunner` registrados.

Paso 11 - Errores comunes del ejercicio

Síntoma	Causa probable	Solución
datos iniciales aparecen duplicados	constructor y runner cargan datos	eliminar carga del constructor
servicio ve repositorio vacío y runner otro distinto	se usó <code>new Repository()</code>	inyectar bean gestionado por Spring
Could not resolve placeholder	propiedad requerida ausente	definirla o proporcionar default
perfil no cambia valores	perfil no está activo o nombre incorrecto	revisar <code>spring.profiles.active</code>
Spring no detecta la config	clase fuera del escaneo o falta <code>@Configuration</code>	revisar paquete/anotación
se exponen secretos en <code>/api/v1/info</code>	endpoint indiscriminado	exponer sólo propiedades no sensibles
se reemplaza <code>ObjectMapper</code> y cambia Jackson	bean creado desde cero	personalizar builder/auto-configuración
inesperadamente aparece inyección por campo	se copió un ejemplo aislado	mantener constructor injection

Después de corregir:

```
./mvnw test
./mvnw -DskipTests package
```

Paso 12 - Reto resuelto: bean personalizado con @Bean

La fuente propone crear un ObjectMapper personalizado con JavaTimeModule para garantizar fechas ISO. Conservamos el objetivo —aprender @Bean y personalización de Jackson— pero actualizamos la implementación para no sustituir innecesariamente el ObjectMapper que Spring Boot ya auto-configura.

Crea:

config/JacksonConfig.java

Contenido:

```
package es.mecd.demo.miproyecto.config;

import com.fasterxml.jackson.databind.SerializationFeature;
import org.springframework.boot.autoconfigure.jackson.Jackson2ObjectMapperBuilderCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JacksonConfig {

    @Bean
    Jackson2ObjectMapperBuilderCustomizer jacksonCustomizer() {
        return builder -> builder.featuresToDisable(
            SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }
}
```

Por qué esta variante

La fuente muestra conceptualmente:

```
@Bean
ObjectMapper objectMapper() {
    return new ObjectMapper()
        .registerModule(new JavaTimeModule());
}
```

Ese código enseña correctamente que @Bean puede registrar un objeto externo, pero Spring Boot ya integra soporte para tipos java.time. La variante con Jackson2ObjectMapperBuilderCustomizer conserva la auto-configuración y personaliza únicamente lo necesario.

Reinicia y comprueba:

```
curl -s http://localhost:8080/api/v1/expedientes/ejemplo
curl -s http://localhost:8080/api/v1/alumnos/1
```

Las fechas deben mantenerse en formato ISO, por ejemplo:

```
"fechaSolicitud": "2025-01-15"
```

Ejecuta finalmente:

```
./mvnw test
./mvnw -DskipTests package
```

Con esto cerramos 2.4: Spring ya no sólo conecta nuestras capas; también controla inicialización, beans adicionales y configuración específica de entorno de forma reproducible.

Comprobaciones adicionales de 2.4

Verificar el perfil desde línea de comandos

Con el JAR empaquetado:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar \
  --spring.profiles.active=dev
```

Consulta:

```
curl -s http://localhost:8080/api/v1/info
```

Detén la aplicación y repite:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar \
  --spring.profiles.active=prod
```

La misma clase Java debe responder con valores de entorno diferentes.

Añadir un perfil de test

Crea:

```
src/test/resources/application-test.properties
```

con:

```
app.entorno=test
app.descripcion=Configuración automática de pruebas
```

Cuando un test necesite explícitamente ese perfil podrá usar:

```
@ActiveProfiles("test")
```

No lo añadas indiscriminadamente a todos los tests: un test unitario de Mockito ni siquiera necesita contexto Spring.

Comprobar que no hay construcción manual de beans

Busca:

```
grep -R "new AlumnoService\|new AlumnoRepository\|new ExpedienteService\|new ExpedienteRepository"
  src/main/java || true
```

En código de producción gestionado por Spring no deberían aparecer esas construcciones para conectar capas.

Verificar el runner sin depender del orden del mapa

Usa JSON, no una comparación textual completa:

```
curl -s http://localhost:8080/api/v1/alumnos > /tmp/alumnos.json
```

Y comprueba conceptualmente:

número de elementos = 2

IDs = 1 y 2

DNIs de demo = DNI-DEMO-01 y DNI-DEMO-02

La comprobación no debería depender del orden interno de un ConcurrentHashMap; para eso el repositorio ya devuelve una lista ordenada.

Punto 2.5 - Introducción al testing

Contexto del ejercicio

En los puntos anteriores construimos tres capas y configuramos Spring. Ahora dejaremos de depender exclusivamente de curl y añadiremos pruebas automatizadas de servicio y controlador.

Paso 1 - Verificar la dependencia de test

Abre pom.xml y confirma que existe:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Esta dependencia agrupa JUnit Jupiter, Mockito, Spring Test, AssertJ y otras utilidades.

Ejecuta:

```
./mvnw dependency:tree
```

Y después:

```
./mvnw test
```

Pregunta

¿Por qué usamos scope test?

Respuesta razonada

Porque esas librerías son necesarias para compilar y ejecutar tests, pero no forman parte de la aplicación que desplegamos en producción.

Paso 2 - Crear la estructura de paquetes de test

Crea:

```
src/test/java/es/mecd/demo/miproyecto/service/
src/test/java/es/mecd/demo/miproyecto/controller/
src/test/java/es/mecd/demo/miproyecto/repository/
```

La estructura refleja la del código de producción para que resulte fácil localizar qué clase prueba cada test.

Paso 3 - Crear el setup de AlumnoServiceTest

Crea:

```
src/test/java/es/mecd/demo/miproyecto/service/AlumnoServiceTest.java
```

Base:

```

package es.mecd.demo.miproyecto.service;

import es.mecd.demo.miproyecto.dto.AlumnoDTO;
import es.mecd.demo.miproyecto.repository.AlumnoRepository;
import org.junit.jupiter.api.BeforeEach;

import static org.mockito.Mockito.mock;

class AlumnoServiceTest {

    private AlumnoRepository repositorio;
    private AlumnoService servicio;

    @BeforeEach
    void setUp() {
        repositorio = mock(AlumnoRepository.class);
        servicio = new AlumnoService(repositorio);
    }
}

```

No usamos @SpringBootTest. Este es un test unitario de una clase Java y su colaborador simulado.

Paso 4 - Test de consulta exitosa

Añade:

```

@Test
void consultarExistenteDevuelveAlumno() {
    AlumnoDTO alumno = new AlumnoDTO(
        "1", "Ana", "García López", "DNI-DEMO-01",
        LocalDate.of(2010, 5, 12), "5º Primaria");

    when(repositorio.buscarPorId("1"))
        .thenReturn(Optional.of(alumno));

    Optional<AlumnoDTO> resultado = servicio.consultar("1");

    assertTrue(resultado.isPresent());
    assertEquals("1", resultado.orElseThrow().getIdificador());
    assertEquals("Ana", resultado.orElseThrow().getNombre());
    verify(repositorio).buscarPorId("1");
}

```

Imports relevantes:

```

import org.junit.jupiter.api.Test;
import java.time.LocalDate;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

```

Paso 5 - Test de consulta no encontrada

```

@Test
void consultarInexistenteDevuelveOptionalVacio() {

```

```

when(repositorio.buscarPorId("999"))
    .thenReturn(Optional.empty());

Optional<AlumnoDTO> resultado = servicio.consultar("999");

assertTrue(resultado.isEmpty());
verify(repositorio).buscarPorId("999");
}

```

Este test protege el contrato `Optional.empty()` y evita una regresión a `null`.

Paso 6 - Test de DNI duplicado

```

@Test
void crearConDniDuplicadoLanzaNegocioExceptionYNoGuarda() {
    AlumnoDTO dto = new AlumnoDTO(
        null, "Otra Ana", "Duplicada", "DNI-DEMO-01",
        LocalDate.of(2010, 1, 1), "5º Primaria");

    when(repositorio.existePorDni("DNI-DEMO-01"))
        .thenReturn(true);

    NegocioException ex = assertThrows(
        NegocioException.class,
        () -> servicio.crear(dto));

    assertTrue(ex.getMessage().contains("DNI-DEMO-01"));
    verify(repositorio, never()).guardar(any());
}

```

La segunda aserción conductual es importante: no basta con lanzar el error; el estado no debe modificarse.

Paso 7 - Test de creación exitosa

La fuente histórica simula `contar()`. Nuestro servicio actual usa una secuencia segura proporcionada por el repositorio.

```

@Test
void crearValidoAsignaIdYGuarda() {
    AlumnoDTO dto = new AlumnoDTO(
        null, "María", "López", "DNI-DEMO-03",
        LocalDate.of(2011, 3, 20), "4º Primaria");

    when(repositorio.existePorDni("DNI-DEMO-03"))
        .thenReturn(false);
    when(repositorio.siguienteIdentificador())
        .thenReturn("3");
    when(repositorio.guardar(any(AlumnoDTO.class)))
        .thenAnswer(invocation -> invocation.getArgument(0));

    AlumnoDTO creado = servicio.crear(dto);

    assertEquals("3", creado.getIdentificador());
    verify(repositorio).siguienteIdentificador();
}

```

```
        verify(repositorio).guardar(any(AlumnoDTO.class));
    }
}
```

Paso 8 - Ejecutar los tests del servicio

Desde consola:

```
./mvnw -Dtest=AlumnoServiceTest test
```

Después toda la suite:

```
./mvnw test
```

Desde IntelliJ IDEA o Eclipse también puedes ejecutar la clase directamente, pero la terminal confirma que la suite no depende de configuración particular del IDE.

Pregunta

¿Por qué conviene comprobar que los tests funcionan con Maven aunque el IDE los muestre verdes?

Respuesta razonada

Porque CI utilizará el build del proyecto, no la configuración local del IDE. Maven es nuestra referencia reproducible.

Paso 9 - Crear AlumnoControllerTest

Crea:

```
src/test/java/es/mecd/demo/miproyecto/controller/AlumnoControllerTest.java
```

Base actualizada para Spring Boot 3.5.x:

```
package es.mecd.demo.miproyecto.controller;

import es.mecd.demo.miproyecto.dto.AlumnoDTO;
import es.mecd.demo.miproyecto.service.AlumnoService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.time.LocalDate;
import java.util.Optional;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(AlumnoController.class)
class AlumnoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private AlumnoService service;
```

```

@Test
void consultarExistenteDevuelve200YJson() throws Exception {
    AlumnoDTO alumno = new AlumnoDTO(
        "1", "Ana", "García López", "DNI-DEMO-01",
        LocalDate.of(2010, 5, 12), "5º Primaria");

    when(service.consultar("1"))
        .thenReturn(Optional.of(alumno));

    mockMvc.perform(get("/api/v1/alumnos/1"))
        .andExpect(status().isOk())
        .andExpect(content().contentTypeCompatibleWith("application/json"))
        .andExpect(jsonPath("$.id").value("1"))
        .andExpect(jsonPath("$.nombre").value("Ana"));
}
}

```

Nota 2026: @MockBean

La fuente original usa @MockBean. No lo hemos olvidado: forma parte de la explicación histórica. En el baseline Spring Boot 3.5.16 está deprecado, por lo que usamos @MockitoBean.

Añade además:

```

@Test
void consultarInexistenteDevuelve404() throws Exception {
    when(service.consultar("999"))
        .thenReturn(Optional.empty());

    mockMvc.perform(get("/api/v1/alumnos/999"))
        .andExpect(status().isNotFound());
}

```

Paso 10 - Añadir test de POST

Añade ObjectMapper del contexto de test:

```

@Autowired
private ObjectMapper objectMapper;

```

Y el test:

```

@Test
void crearDevuelve201LocationYJson() throws Exception {
    AlumnoDTO entrada = new AlumnoDTO(
        null, "María", "López", "DNI-DEMO-03",
        LocalDate.of(2011, 3, 20), "4º Primaria");

    AlumnoDTO creado = new AlumnoDTO(
        "3", "María", "López", "DNI-DEMO-03",
        LocalDate.of(2011, 3, 20), "4º Primaria");

    when(service.crear(any(AlumnoDTO.class)))
        .thenReturn(creado);

    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)

```

```

        .content(objectMapper.writeValueAsString(entrada)))
        .andExpect(status().isCreated())
        .andExpect(header().string(
            "Location", "/api/v1/alumnos/3"))
        .andExpect(jsonPath("$.id").value("3"))
        .andExpect(jsonPath("$.dni").value("DNI-DEMO-03"));
    }

```

Imports:

```

import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.http.MediaType;

import static org.mockito.ArgumentMatchers.any;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;

```

También conviene proteger el 409:

```

@Test
void crearCuandoServicioLanzaNegocioExceptionDevuelve409() throws Exception {
    when(service.crear(any(AlumnoDTO.class)))
        .thenThrow(new NegocioException("DNI duplicado"));

    // perform POST y esperar status().isConflict()
}

```

Paso 11 - Errores comunes del ejercicio

Síntoma	Causa probable	Solución
test del servicio arranca Spring innecesariamente	se usó @SpringBootTest	construir servicio con mock directamente
NullPointerException en service test	mock no preparado	crear mock en @BeforeEach
creación válida recibe ID null	no se configuró siguienteIdentificador()	añadir stubbing
test pasa aunque se guarde tras error	sólo se comprobó excepción	verificar never().guardar(...)
@WebMvcTest no puede crear controlador	falta mock del service	añadir @MockitoBean
se copió @MockBean de la fuente	API deprecada en baseline actual	usar @MockitoBean
POST devuelve 400 en test	JSON o Content-Type incorrectos	usar ObjectMapper y JSON compatible
falla Location	mock devuelve objeto sin ID	devolver recurso creado con ID
test depende del orden de otros tests	estado compartido	reconstruir escenario por test

Ejecuta:

```
./mvnw test
```

No avances mientras haya tests rojos que no estén explicados.

Paso 12 - Reto resuelto: ExpedienteServiceTest

Crea:

src/test/java/es/mecd/demo/miproyecto/service/ExpedienteServiceTest.java

Estructura:

```
class ExpedienteServiceTest {

    private ExpedienteRepository repositorio;
    private ExpedienteService servicio;

    @BeforeEach
    void setUp() {
        repositorio = mock(ExpedienteRepository.class);
        servicio = new ExpedienteService(repositorio);
    }

    @Test
    void consultarExistenteDevuelveExpediente() {
        ExpedienteDTO expediente = /* construir DTO */;
        when(repositorio.buscarPorId("1"))
            .thenReturn(Optional.of(expediente));

        Optional<ExpedienteDTO> resultado = servicio.consultar("1");

        assertTrue(resultado.isPresent());
        assertEquals("1", resultado.orElseThrow().getIdificador());
    }

    @Test
    void consultarInexistenteDevuelveVacio() {
        when(repositorio.buscarPorId("999"))
            .thenReturn(Optional.empty());

        assertTrue(servicio.consultar("999").isEmpty());
    }

    @Test
    void crearSinIdSolicitaIdentificadorYGuarda() {
        ExpedienteDTO dto = /* construir DTO sin ID */;
        when(repositorio.siguienteIdentificador())
            .thenReturn("3");
        when(repositorio.guardar(any(ExpedienteDTO.class)))
            .thenAnswer(inv -> inv.getArgument(0));

        ExpedienteDTO creado = servicio.crear(dto);

        assertEquals("3", creado.getIdificador());
        verify(repositorio).guardar(any(ExpedienteDTO.class));
    }
}
```

Completa la construcción de los DTO con los campos reales de ExpedienteDTO; no sustituyas código ejecutable por pseudocódigo en el proyecto final.

Ejecuta:

```
./mvnw -Dtest=ExpedienteServiceTest test
./mvnw test
```

Al cerrar 2.5 ya tenemos una red de seguridad automática para servicios, controladores y regresiones acumuladas.

Extensión práctica - completar la red de seguridad

Test directo de AlumnoRepository

Crea:

```
src/test/java/es/mecd/demo/miproyecto/repository/AlumnoRepositoryTest.java
```

Incluye al menos estos escenarios:

```
guardar -> consultar
consultar inexistente -> Optional.empty
eliminar existente -> true
eliminar inexistente -> false
modificar copia consultada -> no modifica almacenado
IDs concurrentes -> no se repiten
```

Después:

```
./mvnw -Dtest=AlumnoRepositoryTest test
```

Completar ExpedienteServiceTest con código ejecutable

Un ejemplo completo de creación:

```
@Test
void crearSinIdAsignaIdentificadorYGuarda() {
    ExpedienteDTO dto = new ExpedienteDTO(
        null,
        "Carmen Ruiz",
        "22222222B",
        "NUEVA",
        "BECA",
        LocalDate.of(2025, 2, 1),
        900.0,
        true,
        List.of("Solicitud.pdf"));

    when(repositorio.siguienteIdentificador())
        .thenReturn("3");
    when(repositorio.guardar(any(ExpedienteDTO.class)))
        .thenAnswer(invocation -> invocation.getArgument(0));

    ExpedienteDTO creado = servicio.crear(dto);

    assertEquals("3", creado.getIdentificador());
    assertEquals("Carmen Ruiz", creado.getTitular());
    verify(repositorio).guardar(any(ExpedienteDTO.class));
}
```

Consulta existente:

```
@Test
void consultarExistenteDevuelveExpediente() {
    ExpedienteDTO dto = new ExpedienteDTO(
        "1", "Ana García", "12345678A",
        "EN_TRAMITE", "BECA",
```

```

        LocalDate.of(2025, 1, 15),
        1500.0, true, List.of("DNI.pdf"));

    when(repositorio.buscarPorId("1"))
        .thenReturn(Optional.of(dto));

    ExpedienteDTO resultado = servicio.consultar("1").orElseThrow();

    assertEquals("1", resultado.getIdentificador());
    assertEquals("BECA", resultado.getTipo());
}

```

Consulta inexistente:

```

@Test
void consultarInexistenteDevuelveVacio() {
    when(repositorio.buscarPorId("999"))
        .thenReturn(Optional.empty());

    assertTrue(servicio.consultar("999").isEmpty());
}

```

Ejecuta toda la suite:

```
./mvnw test
```

El objetivo final de 2.5 es que una refactorización posterior de paquetes o calidad en 2.6 pueda hacerse con una red de seguridad real.

Añadir casos web de error

Completa AlumnoControllerTest con el conflicto de negocio:

```

@Test
void crearDuplicadoDevuelve409() throws Exception {
    AlumnoDTO entrada = new AlumnoDTO(
        null, "Otra Ana", "Duplicada", "DNI-DEMO-01",
        LocalDate.of(2010, 1, 1), "5º Primaria");

    when(service.crear(any(AlumnoDTO.class)))
        .thenThrow(new NegocioException(
            "Ya existe un alumno con el DNI DNI-DEMO-01"));

    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(entrada)))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.status").value(409))
        .andExpect(jsonPath("$.error").value("Conflict")));
}

```

JSON mal formado:

```

@Test
void crearConJsonMalFormadoDevuelve400() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"nombre\":\"Pedro\"}"))
        .andExpect(status().isBadRequest());
}

```

Media type incorrecto:

```

@Test
void crearConTextPlainDevuelve415() throws Exception {
    mockMvc.perform(post("/api/v1/alumnos")
        .contentType(MediaType.TEXT_PLAIN)
        .content("hola"))
        .andExpect(status().isUnsupportedMediaType());
}

```

Estos casos convierten en tests automáticos comportamientos que antes sólo comprobábamos con curl.

Comprobar independencia

Ejecuta la suite varias veces y también tests individuales:

```

./mvnw -Dtest=AlumnoServiceTest test
./mvnw -Dtest=AlumnoControllerTest test
./mvnw -Dtest=ExpedienteServiceTest test
./mvnw test

```

Ninguno debe necesitar que otro se haya ejecutado antes.

Punto 2.6 - Buenas prácticas de organización, calidad y documentación

Contexto del ejercicio

Este punto no añade una gran regla de negocio. Su objetivo es convertir el proyecto construido durante M2 en un proyecto más fácil de mantener, probar y entregar.

La fuente contiene 12 pasos reales. Los pasos 11 y 12 quedaron omitidos en su tabla de contenido, pero sí aparecen en el cuerpo y se conservan aquí.

Paso 1 - Revisar la estructura actual

Antes de mover nada, documenta el árbol existente:

```

es.mecd.demo.miproyecto
├── config
├── controller
├── dto
├── exception
├── repository
└── service

```

Y los tests:

```

src/test/java/es/mecd/demo/miproyecto
├── controller
├── repository
└── service

```

Comprueba:

```

./mvnw test
./mvnw -DskipTests package

```

Este resultado será el baseline de la reorganización.

Pregunta

¿Qué dificultad aparecería si controller/ contuviera cincuenta controladores de dominios diferentes?

Respuesta razonada

La capa seguiría siendo visible, pero la funcionalidad quedaría dispersa. Para cambiar “alumnos” tendríamos que navegar por múltiples paquetes globales.

Paso 2 - Identificar problemas de nomenclatura

Busca nombres poco descriptivos.

Revisa especialmente:

métodos genéricos como procesar/hacer/getData
variables de una sola letra fuera de bucles triviales
tests llamados test1/test2
constantes sin significado

Los nombres heredados del curso que ya expresan intención (buscarPorId, crear, actualizarParcial, promocionar, siguienteIdentificador) pueden mantenerse.

No renombres sólo para producir diferencias. Cada renombrado debe mejorar comprensión.

Paso 3 - Añadir Javadoc a las clases principales

Empieza por AlumnoController:

```
/**
 * Expone la API REST de alumnos bajo {@code /api/v1/alumnos}.
 *
 * <p>La clase se limita a traducir entre HTTP y la capa de servicio.</p>
 */
@RestController
@RequestMapping("/api/v1/alumnos")
public class AlumnoController {
```

Añade documentación equivalente a:

AlumnoService
AlumnoRepository
ExpedienteController
ExpedienteService
ExpedienteRepository
DatosInicialesConfig
InfoAplicacionController

El Javadoc no debe narrar implementación trivial. Debe explicar responsabilidad y contrato.

Paso 4 - Añadir Javadoc a métodos públicos

Ejemplo en AlumnoService:

```
/**
 * Consulta un alumno por su identificador.
 */
```

```

* @param id identificador del alumno
* @return alumno encontrado o un Optional vacío
*/
public Optional<AlumnoDTO> consultar(String id) {
    return repositorio.buscarPorId(id);
}

```

Creación:

```

/**
 * Crea un alumno después de validar la unicidad de su DNI.
 *
 * @param dto datos recibidos
 * @return alumno almacenado con identificador
 * @throws NegocioException si ya existe el DNI
 */
public AlumnoDTO crear(AlumnoDTO dto) {
    // ...
}

```

Genera la documentación:

```
./mvnw javadoc:javadoc
```

Corrige errores reales de Javadoc antes de continuar.

Paso 5 - Reorganizar los paquetes por funcionalidad

La fuente propone migrar desde paquetes globales por capa hacia una organización por funcionalidad.

Objetivo:

```

es.mecd.demo.miproyecto
├── alumno
│   ├── AlumnoController.java
│   ├── AlumnoDTO.java
│   ├── AlumnoRepository.java
│   └── AlumnoService.java
├── expediente
│   ├── ExpedienteController.java
│   ├── ExpedienteDTO.java
│   ├── ExpedienteRepository.java
│   ├── ExpedienteService.java
│   └── SolicitanteDTO.java
├── common
│   └── exception
│       └── NegocioException.java
└── config
    ├── DatosInicialesConfig.java
    └── JacksonConfig.java

```

SaludoController puede permanecer en un pequeño paquete web o saludo, porque es una regresión heredada de M0 y no debe desaparecer.

Regla de seguridad

Después de cada grupo de movimientos actualiza package e imports y ejecuta:

```
./mvnw test
```

El contrato HTTP **no cambia** por mover paquetes.

Pregunta

¿Por qué mover una clase de paquete puede romper una aplicación aunque no cambiemos su contenido interno?

Respuesta razonada

Porque cambia su nombre totalmente cualificado. Imports, tests, configuración y referencias deben apuntar al nuevo paquete.

Paso 6 - Añadir Checkstyle a Maven

En pom.xml añade el plugin estable 3.6.0:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-checkstyle-plugin</artifactId>
  <version>3.6.0</version>
  <executions>
    <execution>
      <id>checkstyle</id>
      <phase>verify</phase>
      <goals>
        <goal>check</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

Primera ejecución:

```
./mvnw checkstyle:check
```

Después:

```
./mvnw verify
```

No conviertas cada aviso en una batalla estética. El objetivo es tener un conjunto de reglas útil y estable.

Paso 7 - Ajustar la configuración de Checkstyle

Crea por ejemplo:

config/checkstyle/checkstyle.xml

Configuración mínima pedagógica:

```
<?xml version="1.0"?>
<!DOCTYPE module PUBLIC
  "-//Checkstyle//DTD Checkstyle Configuration 1.3//EN"
  "https://checkstyle.org/dtds/configuration_1_3.dtd">

<module name="Checker">
  <property name="charset" value="UTF-8"/>

  <module name="LineLength">
    <property name="max" value="120"/>
  </module>
```

```

<module name="TreeWalker">
  <module name="TypeName"/>
  <module name="MethodName"/>
  <module name="MemberName"/>
  <module name="JavadocType"/>
</module>
</module>

```

Y referencia el fichero desde el plugin:

```

<configuration>
  <configLocation>config/checkstyle/checkstyle.xml</configLocation>
  <consoleOutput>>true</consoleOutput>
  <failOnViolation>>true</failOnViolation>
</configuration>

```

Ejecuta:

```
./mvnw verify
```

La regla correcta no es “la más estricta posible”, sino la que el equipo entiende y puede mantener.

Paso 8 - Añadir OpenAPI

Añade una propiedad de versión en pom.xml:

```

<properties>
  <java.version>17</java.version>
  <springdoc.version>2.8.13</springdoc.version>
</properties>

```

Y la dependencia:

```

<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>${springdoc.version}</version>
</dependency>

```

La fuente histórica usa 2.3.0. No lo conservamos como dependencia porque Spring Boot 3.5.x corresponde a la línea springdoc 2.8.x.

Arranca y comprueba:

```
curl -i http://localhost:8080/v3/api-docs
```

Abre también:

```
http://localhost:8080/swagger-ui.html
```

Si la versión concreta 2.8.x fijada por el proyecto cambia durante la validación final, el CI será quien determine el valor exacto publicado; la línea de compatibilidad no debe salir de 2.8.x para Boot 3.5.x.

Paso 9 - Añadir anotaciones OpenAPI

En un endpoint de consulta:

```

@Operation(summary = "Consulta un alumno por identificador")
@ApiResponses({
  @ApiResponse(responseCode = "200", description = "Alumno encontrado"),
  @ApiResponse(responseCode = "404", description = "Alumno no encontrado")
})

```

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) {
    // ...
}
```

En creación documenta también 201 y 409:

```
201 -> creado
400 -> JSON inválido
409 -> DNI duplicado
415 -> media type no soportado
```

No conviertas cada método en un bloque enorme de anotaciones redundantes. Añade información que realmente mejore el contrato generado.

Paso 10 - Crear el README del proyecto

Crea:

M2/proyecto/README.md

Contenido mínimo:

```
# Mi Proyecto - Módulo 2
```

```
## Requisitos
```

- Java 17
- Maven Wrapper incluido

```
## Compilar y probar
```

```
```bash
./mvnw test
./mvnw verify
```

## Ejecutar

```
./mvnw spring-boot:run
```

## Perfiles

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

## Endpoints principales

- GET /hola
- GET /adios
- /api/v1/alumnos
- /api/v1/expedientes
- GET /api/v1/info

## Documentación API

- /v3/api-docs
- /swagger-ui.html

## Arquitectura

Controller → Service → Repository

Completa el README con una explicación breve de configuración y tests. Debe permitir arrancar el proyecto

---

### ## Paso 11 - Errores comunes del ejercicio

Síntoma	Causa probable	Solución
imports rotos tras reorganizar	package/import no actualizados	corregir nombres totalmente cualifi
tests dejan de descubrir clases	tests en paquetes antiguos	mover/actualizar tests
Checkstyle falla todo el proyecto	reglas excesivas o configuración incorrecta	empezar por conjunto
Checkstyle no se ejecuta en `verify`	ejecución Maven incompleta	revisar `phase` y `goal`
`/v3/api-docs` devuelve 404	dependencia springdoc ausente/incompatible	usar línea 2.8.x para Boot
Swagger UI no carga	ruta/version/config incorrecta	comprobar starter y `/swagger-ui.html`
README contradice el proyecto	documentación manual olvidada	actualizar con cada cambio relevante
Javadoc falla	comentarios mal formados o referencias inválidas	ejecutar `javadoc:javadoc` y corre
se borran `/hola` o `/adios` al reorganizar	regresión de M0	conservar `SaludoController`
desaparecen contratos de M1/M2	refactor no acumulativo	ejecutar suite + runtime antes de cerrar

Después:

```
```bash
./mvnw test
./mvnw verify
./mvnw javadoc:javadoc
```

Paso 12 - Reto resuelto: common/util y FechasUtil

La fuente cierra M2 con una utilidad transversal de fechas.

Crea:

src/main/java/es/mezd/demo/miproyecto/common/util/FechasUtil.java

Contenido:

```
package es.mezd.demo.miproyecto.common.util;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

/** Utilidades de presentación de fechas para ejemplos del curso. */
public final class FechasUtil {

    private static final DateTimeFormatter FORMATO_DIA_MES_ANIO =
        DateTimeFormatter.ofPattern("dd/MM/yyyy");

    private FechasUtil() {
        throw new IllegalStateException("Clase de utilidad");
    }

    /**
     * Formatea una fecha como dd/MM/yyyy.
     */
}
```

```

    *
    * @param fecha fecha a formatear
    * @return fecha formateada o cadena vacía cuando la fecha es null
    */
    public static String formatear(LocalDate fecha) {
        return fecha == null ? "" : fecha.format(FORMATO_DIA_MES_ANIO);
    }
}

```

Por qué constructor privado

La clase sólo contiene comportamiento estático y no representa un objeto con estado. El constructor privado evita instancias accidentales.

Usar la utilidad sin contaminar el contrato JSON

No cambies `@JsonFormat("yyyy-MM-dd")` de los DTO: el formato ISO sigue siendo el contrato REST.

Puedes usar `FechasUtil` en una representación humana separada, un log pedagógico o un método auxiliar que no sustituya la serialización JSON.

Añade un test:

```

@Test
void formatearDevuelveDiaMesAnio() {
    assertEquals(
        "15/01/2025",
        FechasUtil.formatear(LocalDate.of(2025, 1, 15)));
}

```

Y otro para null:

```

@Test
void formatearNullDevuelveCadenaVacía() {
    assertEquals("", FechasUtil.formatear(null));
}

```

Ejecuta el cierre completo:

```

./mvnw test
./mvnw verify
./mvnw javadoc:javadoc
./mvnw -DskipTests package

```

Con esto M2 queda preparado para su validación final: arquitectura, configuración, tests, organización, calidad, documentación y utilidad común funcionan como un único proyecto acumulativo.

Comprobaciones adicionales de cierre

Añadir plugin de Javadoc si el build lo necesita explícitamente

Si queremos fijar la versión reproducible del plugin, puede declararse:

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-javadoc-plugin</artifactId>
  <version>3.12.0</version>
</plugin>

```

Y ejecutar:

```
./mvnw javadoc:javadoc
```

Verificar OpenAPI automáticamente

Con la aplicación arrancada:

```
curl --fail --silent http://localhost:8080/v3/api-docs \
  > /tmp/openapi.json
```

Comprueba que el JSON contiene rutas acumulativas como:

```
/api/v1/alumnos
/api/v1/expedientes
/api/v1/info
```

Verificar Swagger UI

```
curl -I http://localhost:8080/swagger-ui.html
```

Puede existir una redirección hacia la UI interna; lo importante es que el recurso sea alcanzable con la dependencia configurada.

Gate manual final del alumno

Antes de considerar terminado M2:

```
./mvnw clean test
./mvnw verify
./mvnw javadoc:javadoc
./mvnw -DskipTests package
```

Y con el JAR:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar
```

Comprueba al menos:

```
/hola
/adios
/api/v1/alumnos
/api/v1/expedientes/ejemplo
/api/v1/info
/v3/api-docs
/swagger-ui.html
```

Este cierre garantiza que organización y herramientas de calidad no hayan destruido funcionalidad aprendida en módulos anteriores.

Opcional: inspeccionar SpotBugs sin convertirlo todavía en gate duro

Si el plugin queda configurado en el proyecto final, ejecuta:

```
./mvnw spotbugs:check
```

Revisa cada hallazgo antes de decidir si debe bloquear el build. En esta primera introducción, Checkstyle sí formará parte del gate obligatorio; SpotBugs puede mantenerse como análisis adicional hasta estabilizar sus reglas.

Comprobación del README desde cero

Haz una prueba sencilla: imagina una máquina nueva con Java 17 y Git. Siguiendo únicamente README.md deberías poder llegar a:

```
./mvnw test
./mvnw spring-boot:run
```

Si necesitas información que sólo existe en una conversación o en tu memoria, el README todavía está incompleto.